

Clustering jerárquico y k-means V2

Panorama general

En esta parte empezamos a trabajar con **agrupamientos**.

La pregunta central es:

¿Puedo formar grupos de observaciones parecidas y defender que esos grupos tienen sentido?

En unidades anteriores usamos PCA, PCoA o NMDS para ver estructura en los datos. Ahora damos un paso más: intentamos convertir parte de esa estructura en grupos concretos.

Hay dos caminos principales.

Clustering jerárquico construye una historia de uniones. Empieza con cada observación separada y las va uniando según su parecido. El resultado es un dendrograma. Después decidimos dónde cortar ese árbol.

Clustering no jerárquico no construye un árbol. Busca directamente una partición. El ejemplo principal es k-means: primero elegimos cuántos grupos queremos y después el algoritmo intenta formarlos.

Clustering jerárquico:

observaciones separadas → uniones sucesivas → dendrograma → corte

K-means:

elijo k → centroides → asignación de puntos → grupos finales

En este recorrido vamos a separar dos cosas que muchas veces aparecen mezcladas en los scripts:

Método estadístico:

lo que calcula el agrupamiento.

Visualización:

gráficos que ayudan a ver mejor, pero no reemplazan al método.

Esto ayuda a no confundir una función de gráfico con el análisis principal.

Preparación de datos

Vamos a trabajar primero con `USArrests`. Cada fila es un estado de Estados Unidos. Las variables son medidas de criminalidad y urbanización.

```
data(USArrests)

dim(USArrests)
```

```
[1] 50  4
```

```
head(USArrests)
```

	Murder	Assault	UrbanPop	Rape
Alabama	13.2	236	58	21.2
Alaska	10.0	263	48	44.5
Arizona	8.1	294	80	31.0
Arkansas	8.8	190	50	19.5
California	9.0	276	91	40.6
Colorado	7.9	204	78	38.7

```
summary(USArrests)
```

Murder	Assault	UrbanPop	Rape
Min. : 0.800	Min. : 45.0	Min. : 32.00	Min. : 7.30
1st Qu.: 4.075	1st Qu.: 109.0	1st Qu.: 54.50	1st Qu.: 15.07
Median : 7.250	Median : 159.0	Median : 66.00	Median : 20.10
Mean : 7.788	Mean : 170.8	Mean : 65.54	Mean : 21.23
3rd Qu.: 11.250	3rd Qu.: 249.0	3rd Qu.: 77.75	3rd Qu.: 26.18
Max. : 17.400	Max. : 337.0	Max. : 91.00	Max. : 46.00

Las variables no están en la misma escala. Por eso las estandarizamos antes de calcular distancias o aplicar k-means.

```
USArrests_scale <- scale(USArrests)
```

La idea es simple: si una variable tiene diferencias numéricas mucho más grandes, puede dominar la distancia. El escalado evita eso.

Distancias

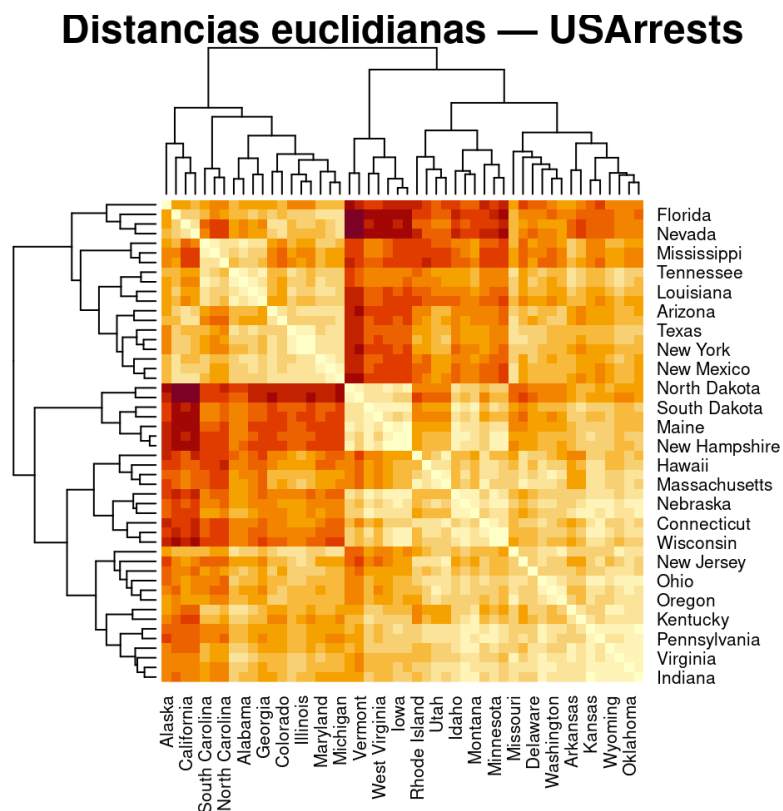
El clustering jerárquico parte de una matriz de distancias. Primero tenemos que decidir qué significa “parecido”.

```
dist_euc <- dist(USArrests_scale, method = "euclidean")
dist_manh <- dist(USArrests_scale, method = "manhattan")
```

La distancia euclidiana es la más habitual para datos numéricos escalados. Manhattan también puede usarse, y suele ser menos sensible a diferencias extremas.

Una forma básica de mirar la matriz de distancias es con un heatmap.

```
heatmap(as.matrix(dist_euc),  
        symm = TRUE,  
        main = "Distancias euclidianas — USArrests")
```



El orden visual importa mucho. Si las filas quedan en orden alfabético, puede no verse ningún patrón. Cuando el heatmap reordena por similitud, pueden aparecer bloques.

Clustering jerárquico

El clustering jerárquico empieza con cada observación separada y va uniendo observaciones o grupos.

La secuencia es:

datos escalados → matriz de distancias → linkage → dendrograma → corte

- - -

Linkage

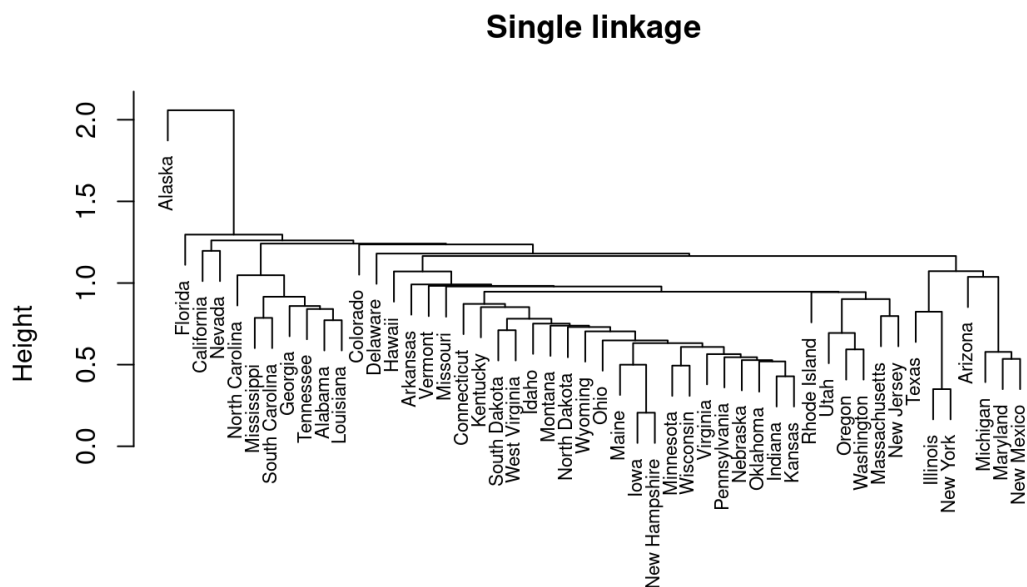
Cuando ya se formaron grupos, necesitamos decidir cómo medir la distancia entre un grupo y otro. Eso es el **linkage**.

single = mínima distancia entre grupos
complete = máxima distancia entre grupos
average = promedio de distancias entre grupos
Ward.D2 = une grupos minimizando el aumento de variabilidad interna

```
hc_single <- hclust(dist_euc, method = "single")
hc_complete <- hclust(dist_euc, method = "complete")
hc_average <- hclust(dist_euc, method = "average")
hc_ward <- hclust(dist_euc, method = "ward.D2")
```

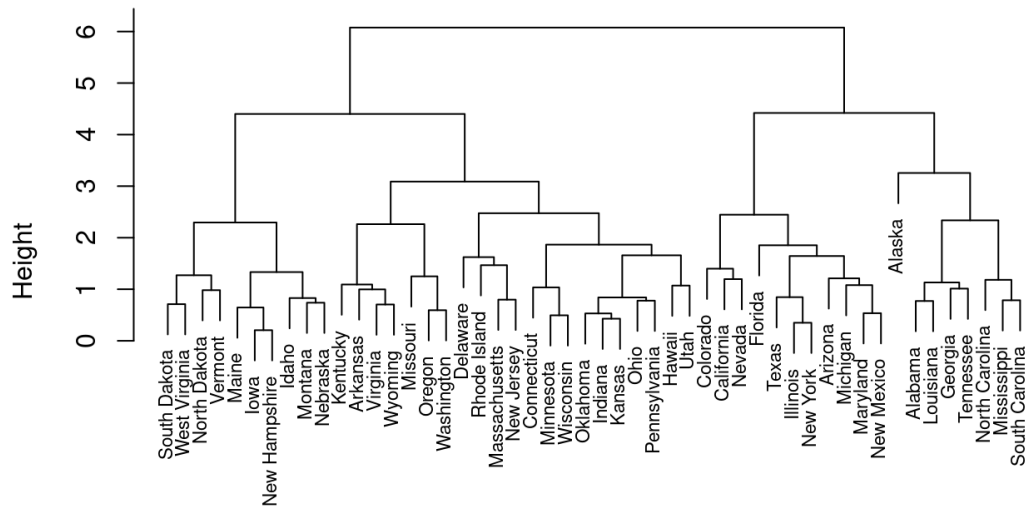
Comparamos los dendrogramas.

```
plot(hc_single, main = "Single linkage", sub = "", xlab = "", cex =
```



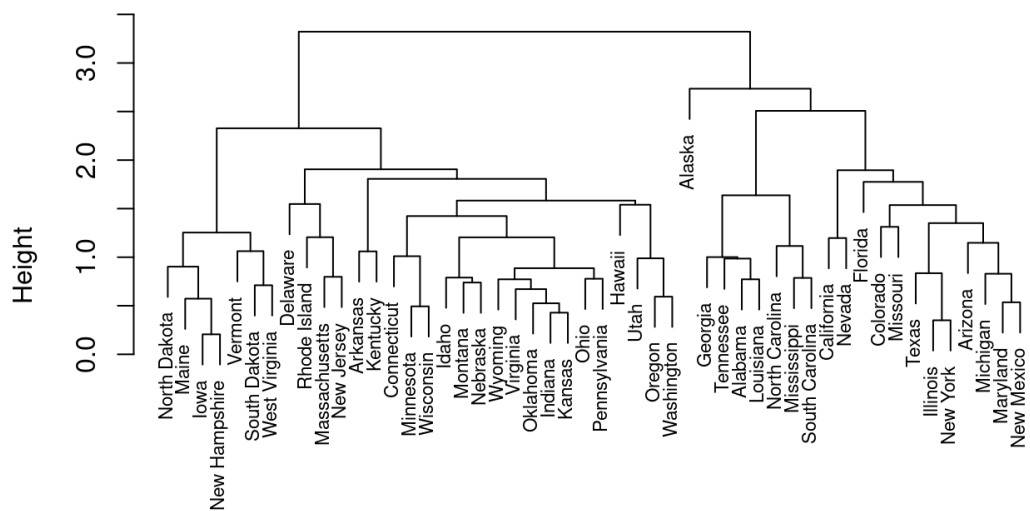
```
plot(hc_complete, main = "Complete linkage", sub = "", xlab = "", cex =
```

Complete linkage

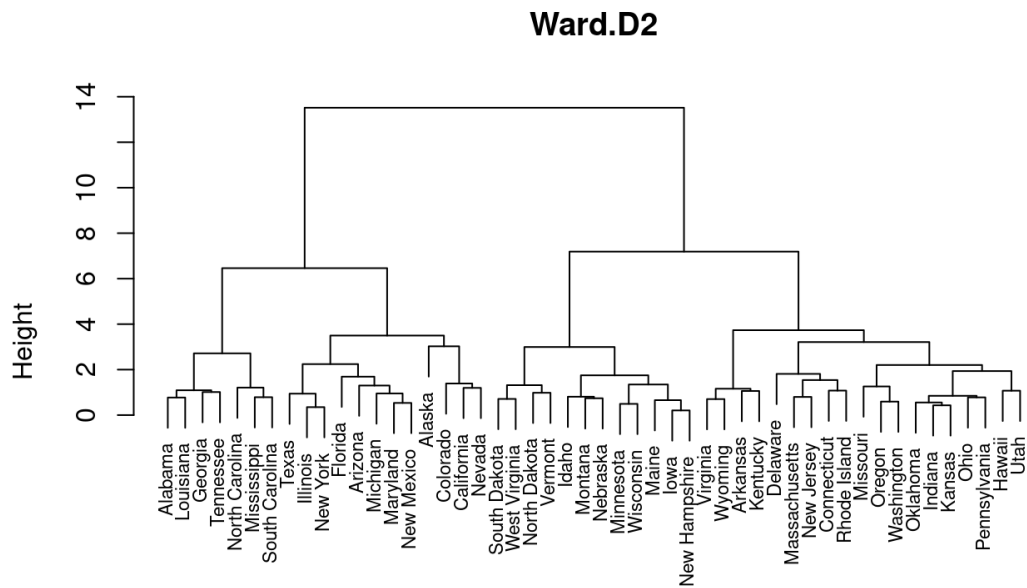


```
plot(hc_average, main = "Average linkage", sub = "", xlab = "", cex =
```

Average linkage



```
plot(hc_ward, main = "Ward.D2", sub = "", xlab = "", cex =
```



El dendrograma no es una verdad única. Cambia si cambiamos la distancia o el linkage. Ward suele generar grupos compactos y de tamaño más parejo, por eso muchas veces es una buena opción inicial.

Corte del dendrograma

El dendrograma muestra toda la historia de uniones. Para obtener grupos concretos hay que cortarlo.

```
grupos_ward4 <- cutree(hc_ward, k = 4)
```

```
table(grupos_ward4)
```

grupos_ward4

```
1  2  3  4
7 12 19 12
```

```
head(grupos_ward4)
```

Alabama	Alaska	Arizona	Arkansas	California	Colorado
1	2	2	3	2	2

`cutree()` convierte el árbol en una partición. En este caso pedimos 4 grupos.

Silueta por observación

Antes de usar la silueta para elegir k , conviene entender qué mide en una observación.

Para cada observación, la silueta compara dos ideas:

- a = qué tan lejos está de los puntos de su propio grupo
- b = qué tan lejos está del grupo vecino más cercano

La lectura es:

- cerca de 1 = bien asignada
- cerca de 0 = está en una frontera
- negativa = podría encajar mejor en otro grupo

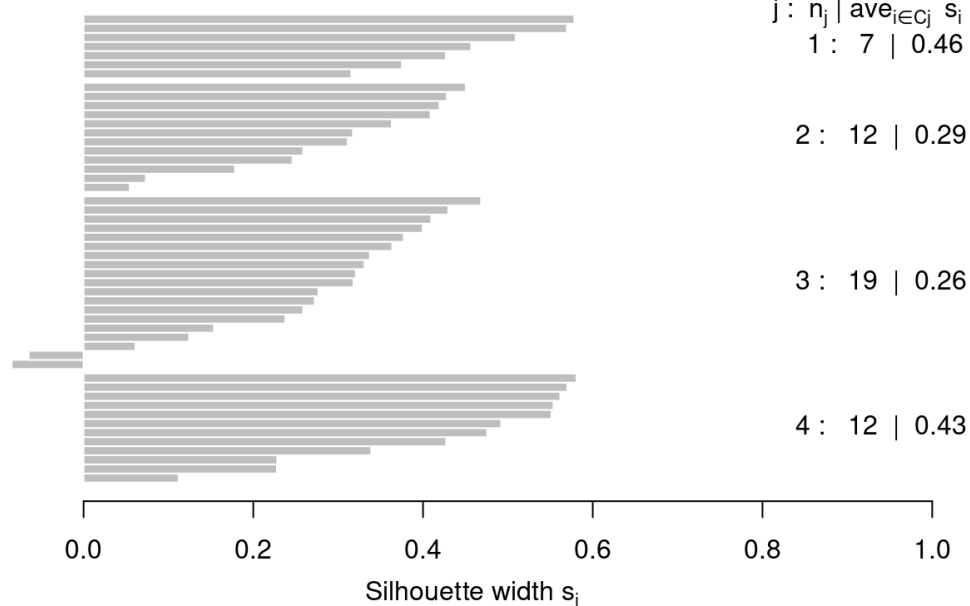
Primero miramos la silueta para una partición concreta. En este caso usamos el corte con $k = 4$.

```
library(cluster)

sil_4 <- silhouette(grupos_ward4, dist_euc)
plot(sil_4, main = "Silueta por observación — Ward k = 4")
```

Silueta por observación — Ward k = 4

n = 50



Average silhouette width : 0.34

Este gráfico no elige k . Sirve para revisar, dentro de una solución ya elegida, qué observaciones quedaron bien asignadas y cuáles son dudosas.

observaciones quedaron bien asignadas y cuáles son dudosas.

Elección de la cantidad de grupos

Después de entender la silueta individual, podemos usar promedios para comparar distintas cantidades de grupos.

No hay una única regla. Usamos varios apoyos.

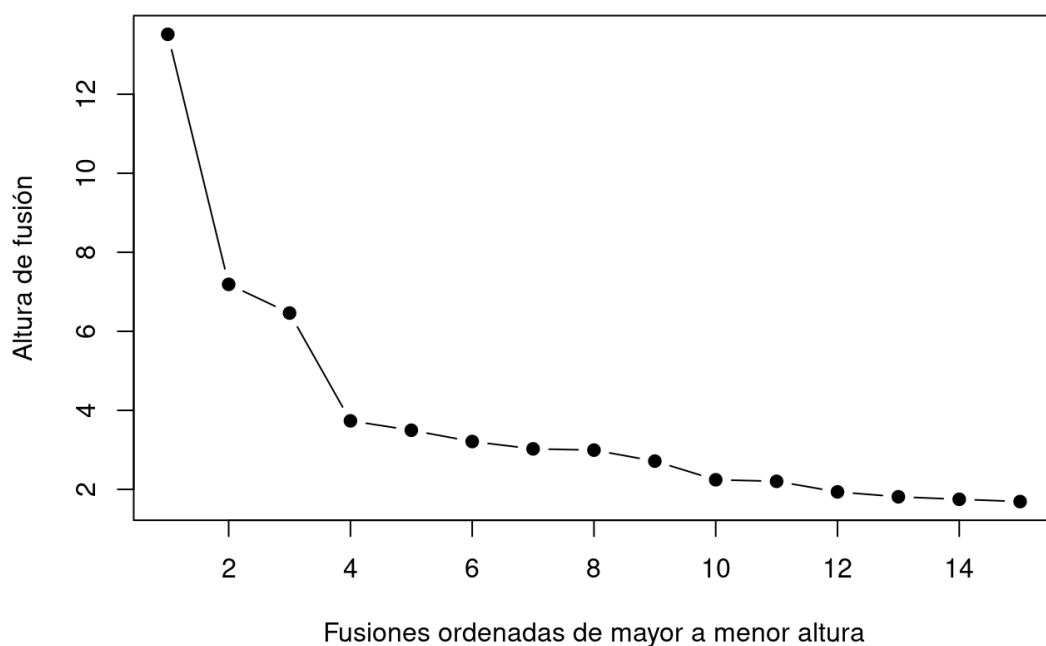
Alturas de fusión

Cada unión del dendrograma ocurre a una altura. Una altura grande indica que se están juntando grupos más distintos.

```
heights <- sort(hc_ward$height, decreasing = TRUE)

n_max <- 15
plot(1:n_max, heights[1:n_max],
     type = "b",
     pch = 19,
     xlab = "Fusiones ordenadas de mayor a menor altura",
     ylab = "Altura de fusión",
     main = "Scree plot de alturas — Ward")
```

Scree plot de alturas — Ward

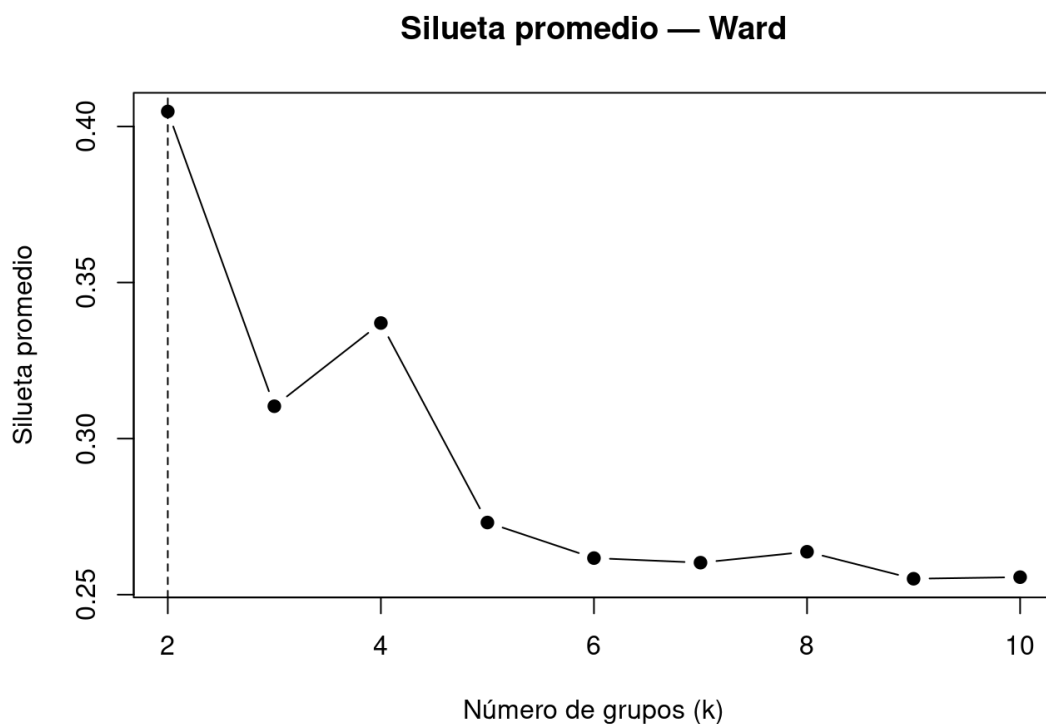


Buscamos saltos grandes. El corte suele ubicarse antes de empezar a unir grupos demasiado distintos.

Silueta promedio para comparar k

Ahora repetimos el cálculo para varios valores de k y guardamos un solo resumen: el promedio de las siluetas individuales.

```
sil_ward <- sapply(2:10, function(k) {  
  grupos <- cutree(hc_ward, k = k)  
  sil <- silhouette(grupos, dist_euc)  
  mean(sil[, 3])  
})  
  
plot(2:10, sil_ward,  
     type = "b",  
     pch = 19,  
     xlab = "Número de grupos (k)",  
     ylab = "Silueta promedio",  
     main = "Silueta promedio — Ward")  
  
abline(v = which.max(sil_ward) + 1, lty = 2)
```



La silueta promedio ayuda a comparar particiones. Los criterios pueden no coincidir, eso no es un error. Miren cosas distintas.

coincidir, eso no es un error. Miran cosas distintas.

Visualizaciones opcionales

Estas funciones son útiles para presentar, pero no reemplazan al cálculo principal.

```
library(factoextra)
```

Cargando paquete requerido: ggplot2

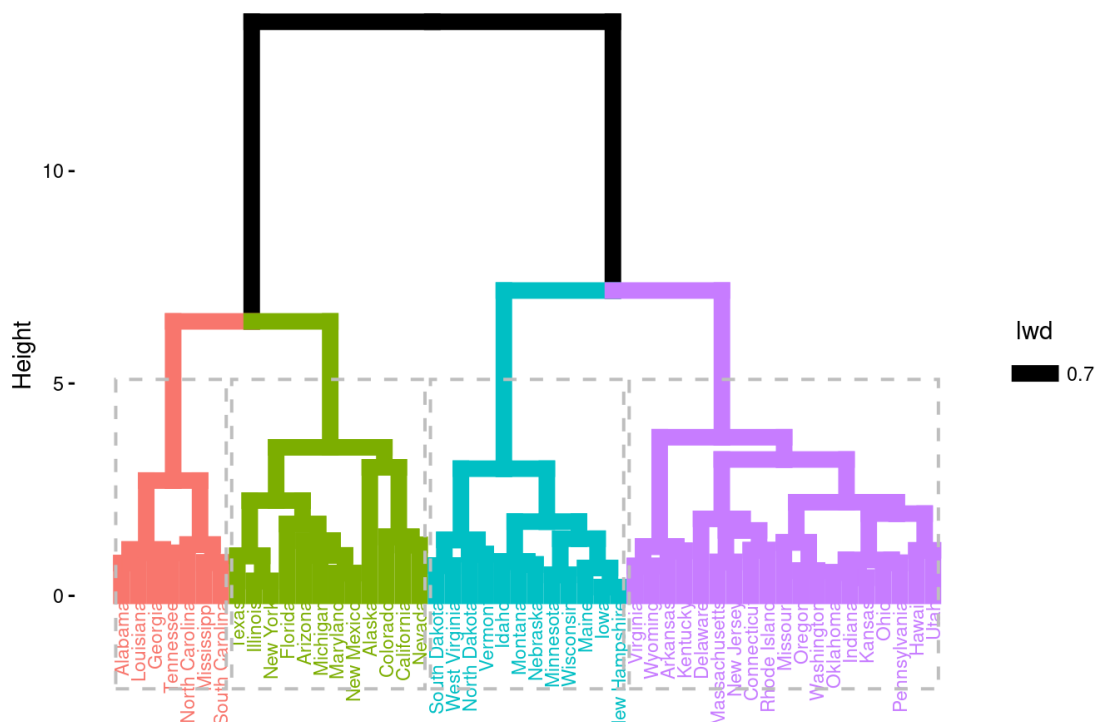
Welcome to factoextra!

Want to learn more? See two factoextra-related books at <https://www.datanovia.com/en/product/practical-guide-to-principal-component-methods-in-r/>

```
library(ggplot2)

fviz_dend(hc_ward,
  k = 4,
  rect = TRUE,
  cex = 0.6,
  main = "Dendrograma Ward - k = 4")
```

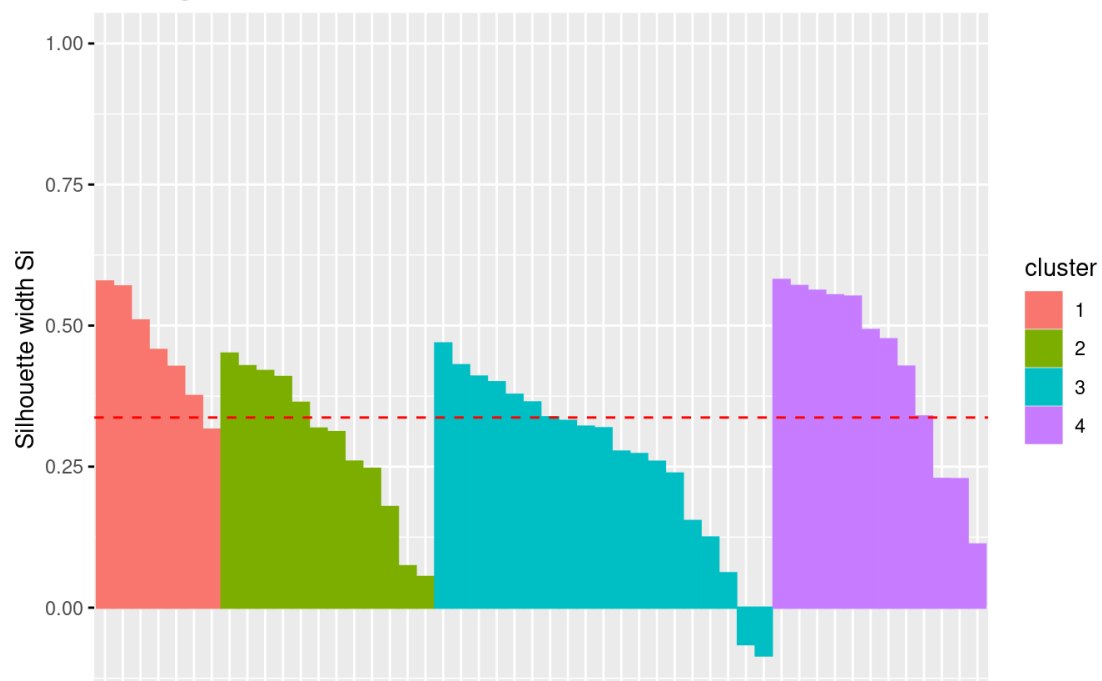
Dendrograma Ward — k = 4



```
fviz_silhouette(sil_4)
```

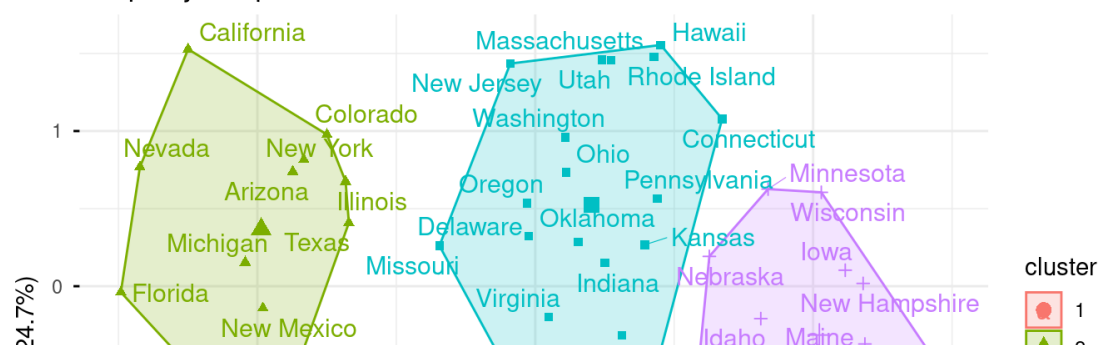
	cluster	size	ave.sil.width
1	1	7	0.46
2	2	12	0.29
3	3	19	0.26
4	4	12	0.43

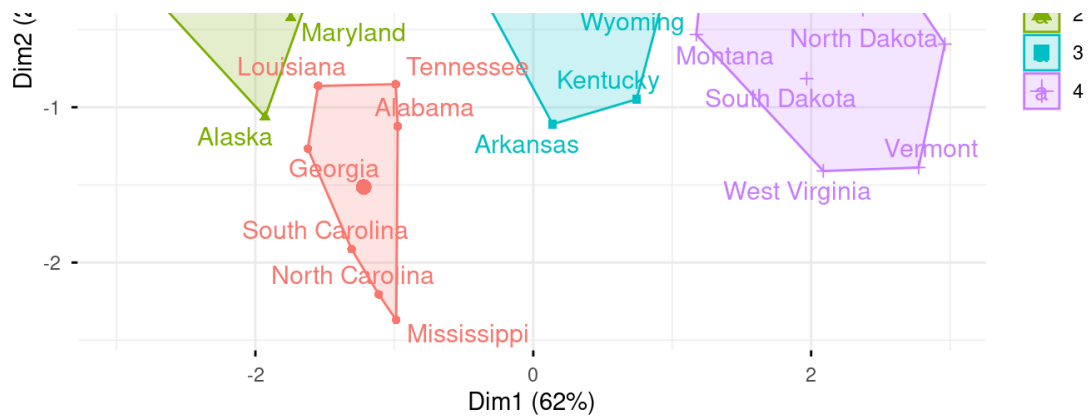
Clusters silhouette plot
Average silhouette width: 0.34



```
fviz_cluster(list(data = USArrests_scale, cluster = grupos_ward4),
  ellipse.type = "convex",
  repel = TRUE,
  ggtheme = theme_minimal(),
  main = "Grupos jerárquicos – Ward k = 4")
```

Grupos jerárquicos — Ward k = 4





`fviz_cluster()` proyecta los datos en dos dimensiones para poder verlos. Los grupos ya fueron calculados antes.

Comparación de dendrogramas

Podemos comparar dos dendrogramas para ver cuánto cambia la estructura al cambiar el linkage.

```
library(dendextend)
```

```
-----
```

```
Welcome to dendextend version 1.19.1
```

```
Type citation('dendextend') for how to cite the package.
```

```
Type browseVignettes(package = 'dendextend') for the package vignette.
```

```
The github page is: https://github.com/talgalili/dendextend/
```

```
Suggestions and bug-reports can be submitted at: https://github.com/
talgalili/dendextend/issues
```

```
You may ask questions at stackoverflow, use the r and dendextend tags:
https://stackoverflow.com/questions/tagged/dendextend
```

```
To suppress this message use:
```

```
suppressPackageStartupMessages(library(dendextend))
```

```
-----
```

```
Adjuntando el paquete: 'dendextend'
```

```
The following object is masked from 'package:stats':
```

```
cutree
```

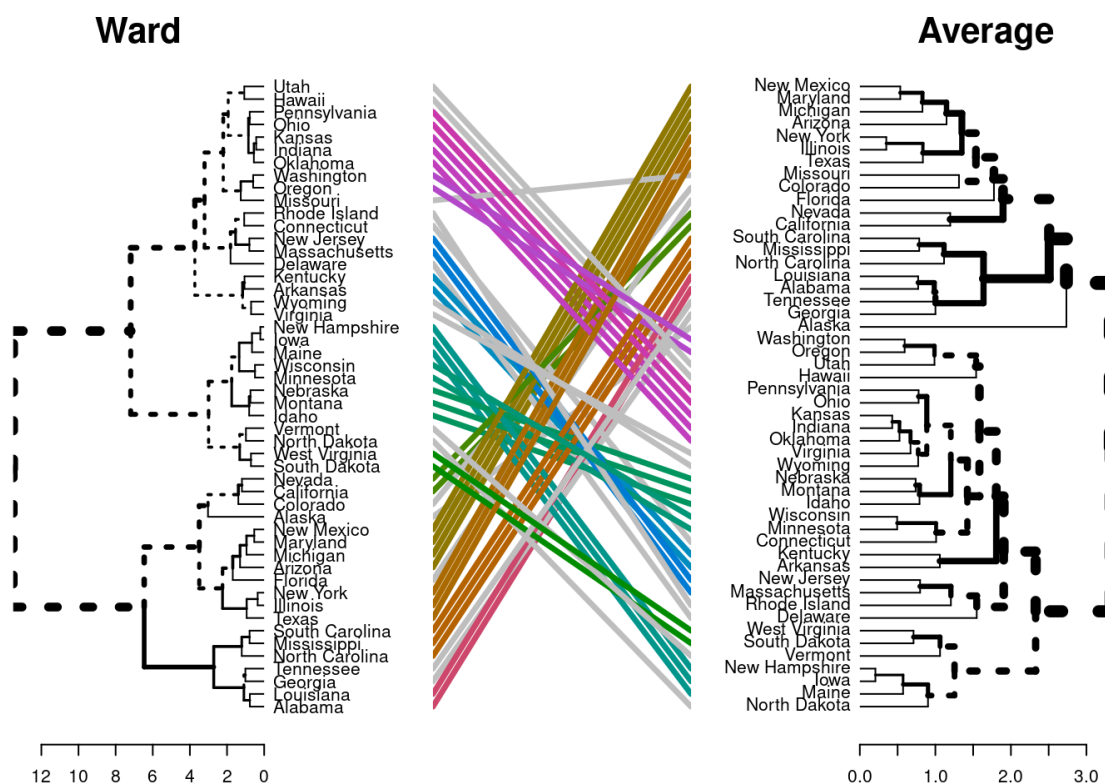
```
dend1 <- as.dendrogram(hc1 <- hclust(d1, method = "ward.D"))
```

```
dend_ward <- as.dendrogram(hc_ward)
dend_avg <- as.dendrogram(hc_average)
```

El tanglegram conecta las mismas observaciones en dos dendrogramas. Muchas líneas cruzadas indican más diferencias.

```
dendextend::tanglegram(dend_ward, dend_avg,
  main_left = "Ward",
  main_right = "Average",
  margin_inner = 8)
```

Loading required namespace: colorspace



También podemos resumir la similitud con una correlación cofenética.

```
cor_cophenetic(dend_ward, dend_avg)
```

```
[1] 0.843143
```

Valores cercanos a 1 indican dendrogramas más parecidos.

K-means

K-means no construye un árbol. Hay que definir antes cuántos grupos queremos

formar.

La idea es:

1. Se colocan k centroides iniciales.
2. Cada observación se asigna al centroide más cercano.
3. Se recalcula cada centroide como promedio de su grupo.
4. El proceso se repite hasta que las asignaciones se estabilizan.

El objetivo es formar grupos compactos alrededor de centroides.

Ejemplo con datos simulados

```
library(MASS)

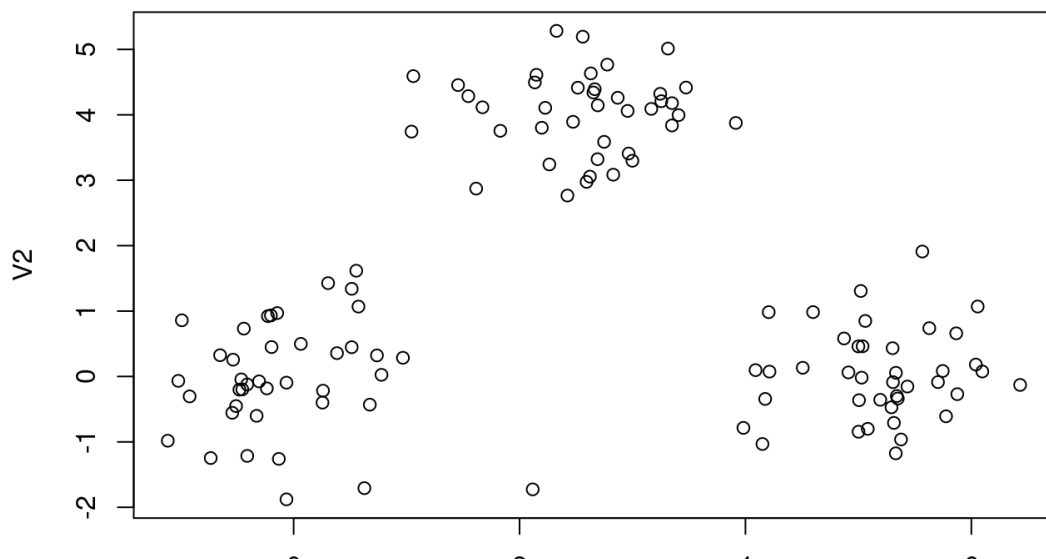
set.seed(42)

grupo_sim <- c(rep("A", 40), rep("B", 40), rep("C", 40))

X_sim <- rbind(
  mvrnorm(40, mu = c(0, 0), Sigma = diag(0.5, 2)),
  mvrnorm(40, mu = c(5, 0), Sigma = diag(0.5, 2)),
  mvrnorm(40, mu = c(2.5, 4), Sigma = diag(0.5, 2))
)

colnames(X_sim) <- c("V1", "V2")

plot(X_sim)
```





```
km3 <- kmeans(X_sim,  
              centers = 3,  
              nstart = 25,  
              iter.max = 100)
```

```
km3$centers
```

	V1	V2
1	-0.05645894	-0.02795610
2	5.18257473	0.03439847
3	2.56666081	4.02250124

```
km3$size
```

```
[1] 40 40 40
```

Los centroides son los centros promedio finales de cada grupo.

Sumas de cuadrados

```
km3$withinss
```

```
[1] 45.50005 32.43469 32.21189
```

```
km3$tot.withinss
```

```
[1] 110.1466
```

```
km3$betweenss
```

```
[1] 979.8172
```

```
km3$totss
```

```
[1] 1089.964
```

Lectura:

withinss = dispersión interna de cada grupo

tot.withinss = dispersión interna total
betweenss = variabilidad entre grupos
totss = variabilidad total

La relación es:

```
km3$totss
```

```
[1] 1089.964
```

```
km3$tot.withinss + km3$betweenss
```

```
[1] 1089.964
```

Si la partición es buena, esperamos grupos compactos y separados:

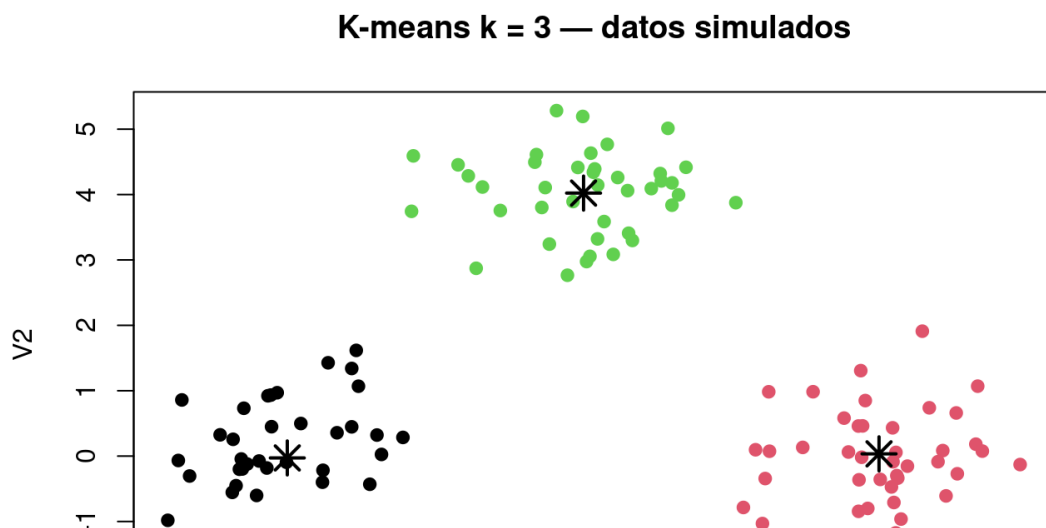
tot.withinss bajo
betweenss alto

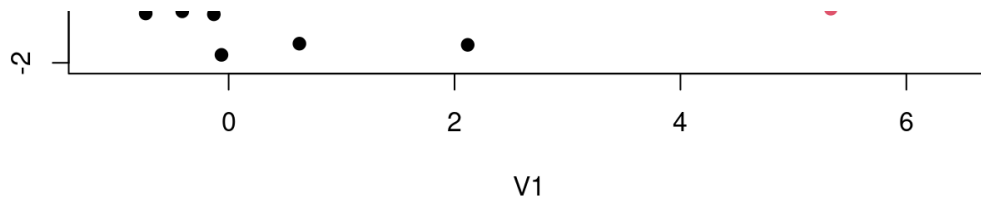
El porcentaje de variabilidad explicada por la separación entre grupos es:

```
VE <- km3$betweenss / km3$totss * 100  
VE
```

```
[1] 89.89447
```

```
plot(X_sim,  
     col = km3$cluster,  
     pch = 19,  
     main = "K-means k = 3 — datos simulados")  
points(km3$centers, pch = 8, cex = 2, lwd = 2)
```





K-means en USArrests

Efecto del escalado

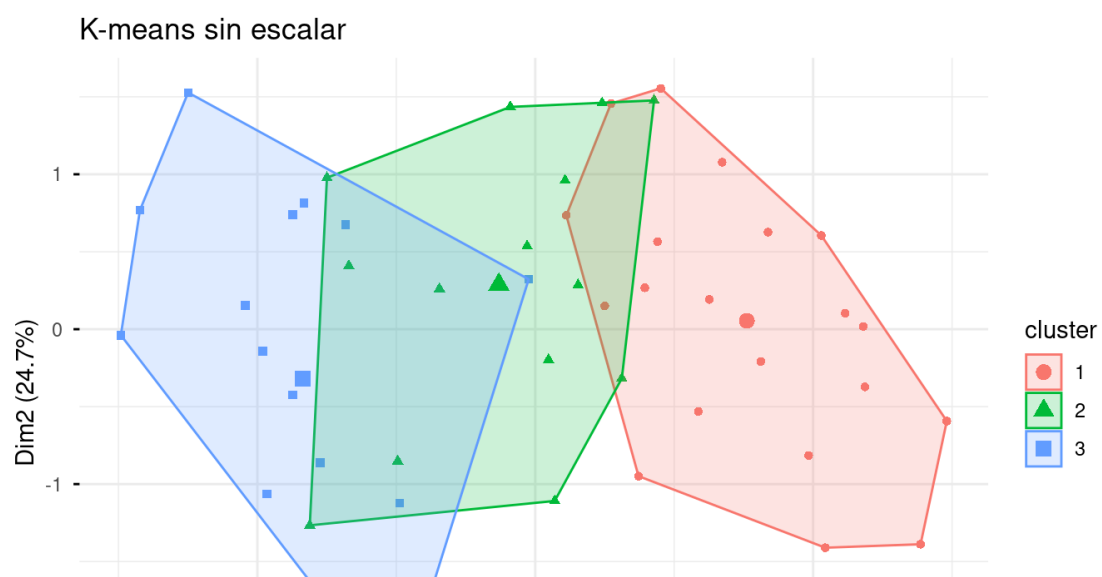
Comparamos k-means con datos originales y escalados.

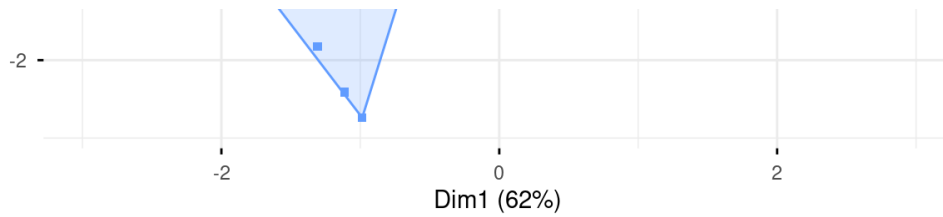
```
set.seed(42)

km_sin_escalar <- kmeans(USArrests,
  centers = 3,
  nstart = 25)

km_escalado <- kmeans(USArrests_scale,
  centers = 3,
  nstart = 25)
```

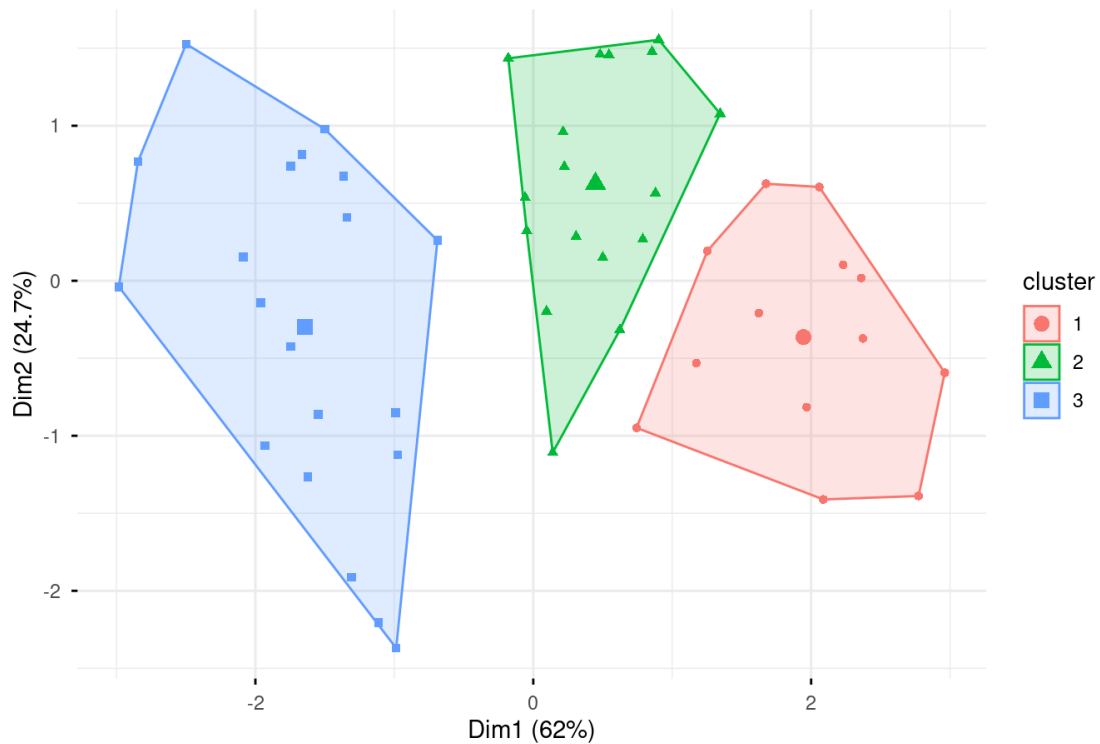
```
fviz_cluster(km_sin_escalar,
  data = USArrests,
  geom = "point",
  ggtheme = theme_minimal(),
  main = "K-means sin escalar")
```





```
fviz_cluster(km_escalado,
  data = USArrests_scale,
  geom = "point",
  ggtheme = theme_minimal(),
  main = "K-means con datos escalados")
```

K-means con datos escalados



Sin escalar, una variable con mayor magnitud puede dominar la distancia. Con datos escalados, todas las variables aportan en escala comparable.

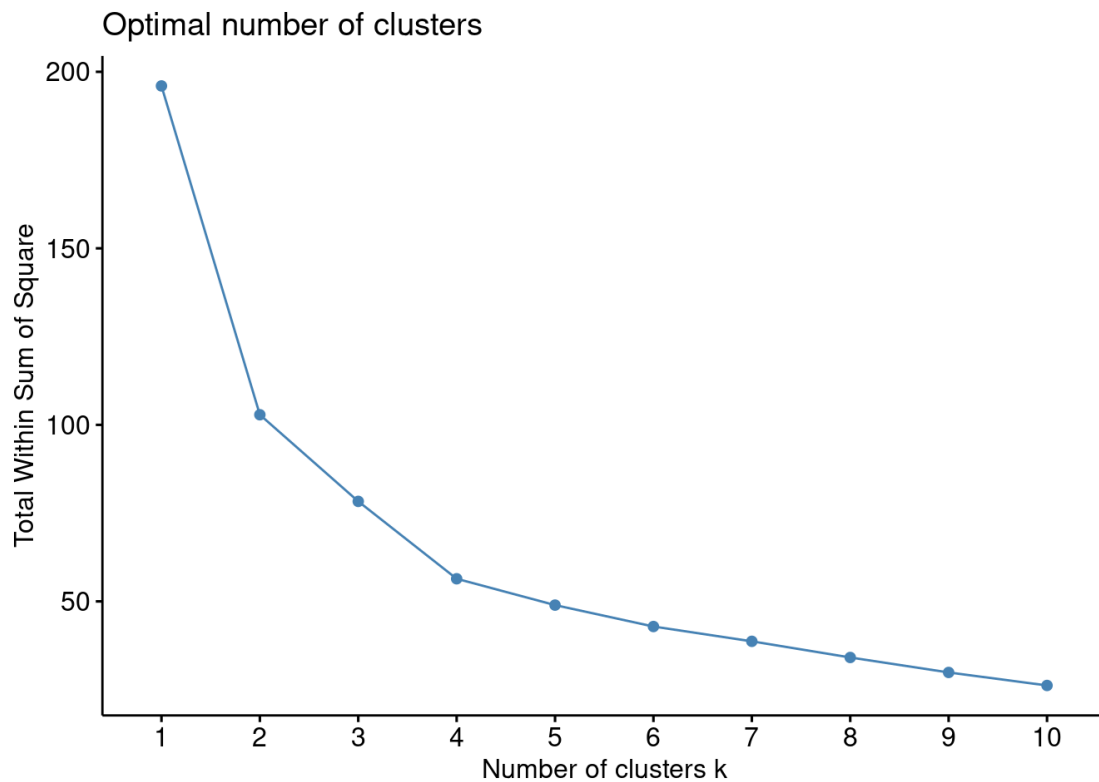
Elección de k

En k-means debemos elegir k antes. Usamos varios apoyos.

Método del codo

```
fviz_nbclust(USArrests_scale,
```

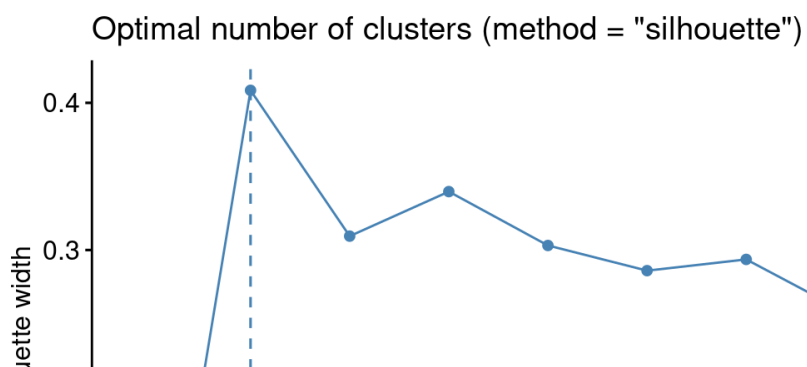
```
kmeans,  
method = "wss",  
k.max = 10,  
nstart = 25)
```

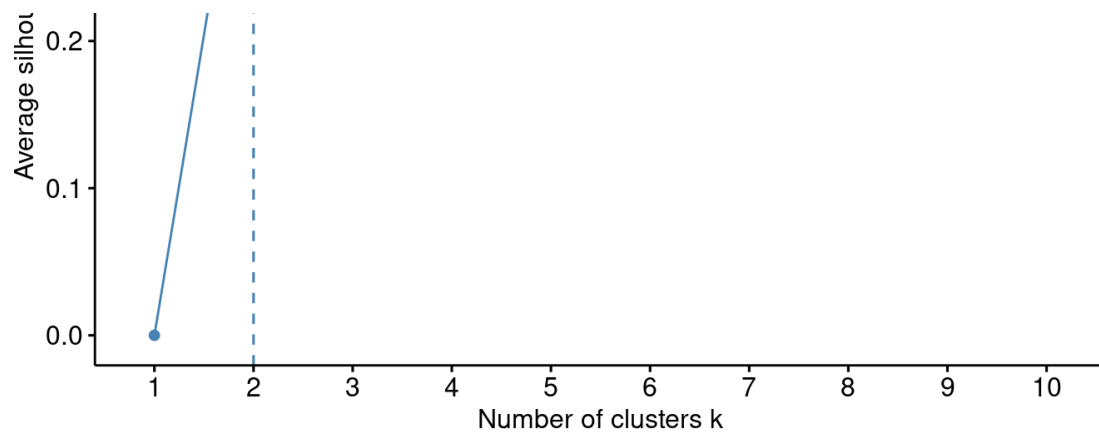


La WCSS baja siempre cuando aumenta k . Buscamos el punto donde la mejora empieza a ser pequeña.

Silueta

```
fviz_nbclust(USArrests_scale,  
kmeans,  
method = "silhouette",  
k.max = 10,  
nstart = 25)
```





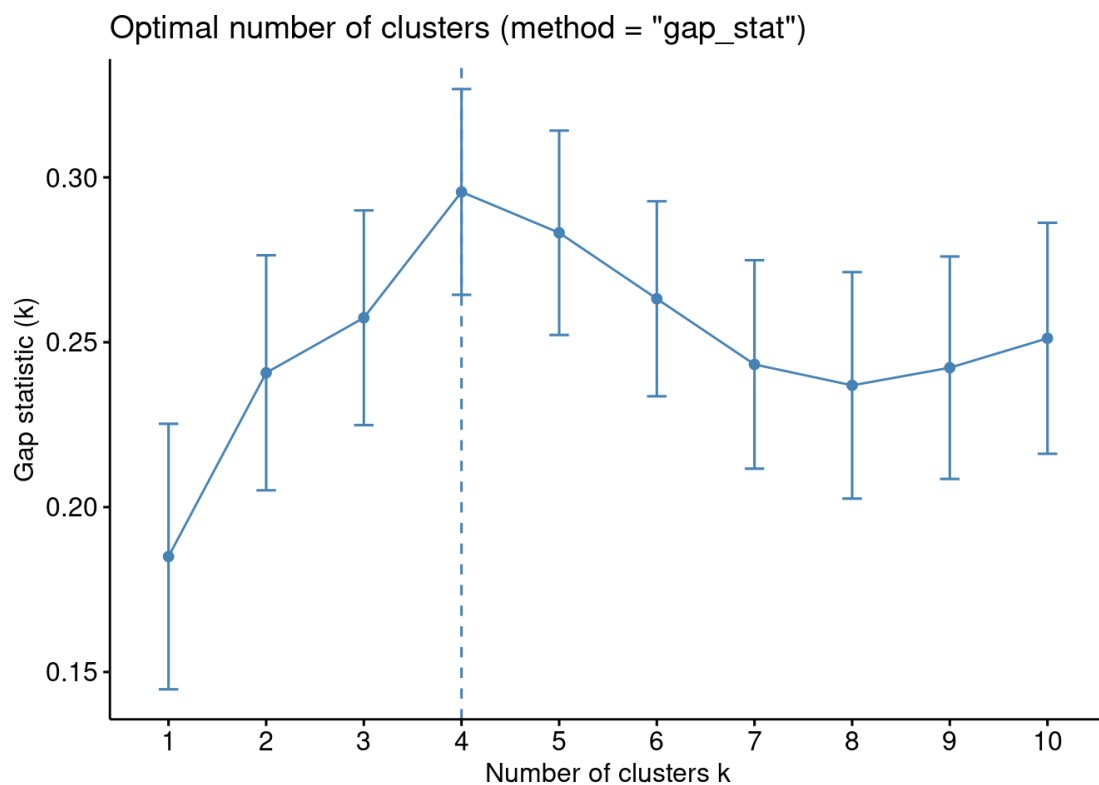
La silueta mira qué tan bien asignadas quedan las observaciones.

Gap statistic

```
set.seed(42)

gap_stat <- clusGap(USArrests_scale,
  FUN = kmeans,
  nstart = 25,
  K.max = 10,
  B = 50)

fviz_gap_stat(gap_stat)
```



Gap compara la estructura observada contra datos sin estructura.

K-means final con $k = 4$

Después de comparar criterios, usamos $k = 4$.

```
set.seed(42)

km_final <- kmeans(USArrests_scale,
                  centers = 4,
                  nstart = 25)

km_final$size
```

```
[1] 13 16  8 13
```

```
km_final$centers
```

	Murder	Assault	UrbanPop	Rape
1	-0.9615407	-1.1066010	-0.9301069	-0.96676331
2	-0.4894375	-0.3826001	0.5758298	-0.26165379
3	1.4118898	0.8743346	-0.8145211	0.01927104
4	0.6950701	1.0394414	0.7226370	1.27693964

Los centroides están estandarizados:

0 = promedio general
+ = por encima del promedio
- = por debajo del promedio

Con estos centroides podemos describir perfiles:

Grupo 1: bajo en todas las variables.
Grupo 2: urbanización alta, criminalidad baja o moderada.
Grupo 3: homicidios altos, urbanización baja.
Grupo 4: urbanización y criminalidad altas.

Gráfico de centroides

```
centroid_df <- as.data.frame(km_final$centers)
centroid_df$Grupo <- paste0("G", 1:4)

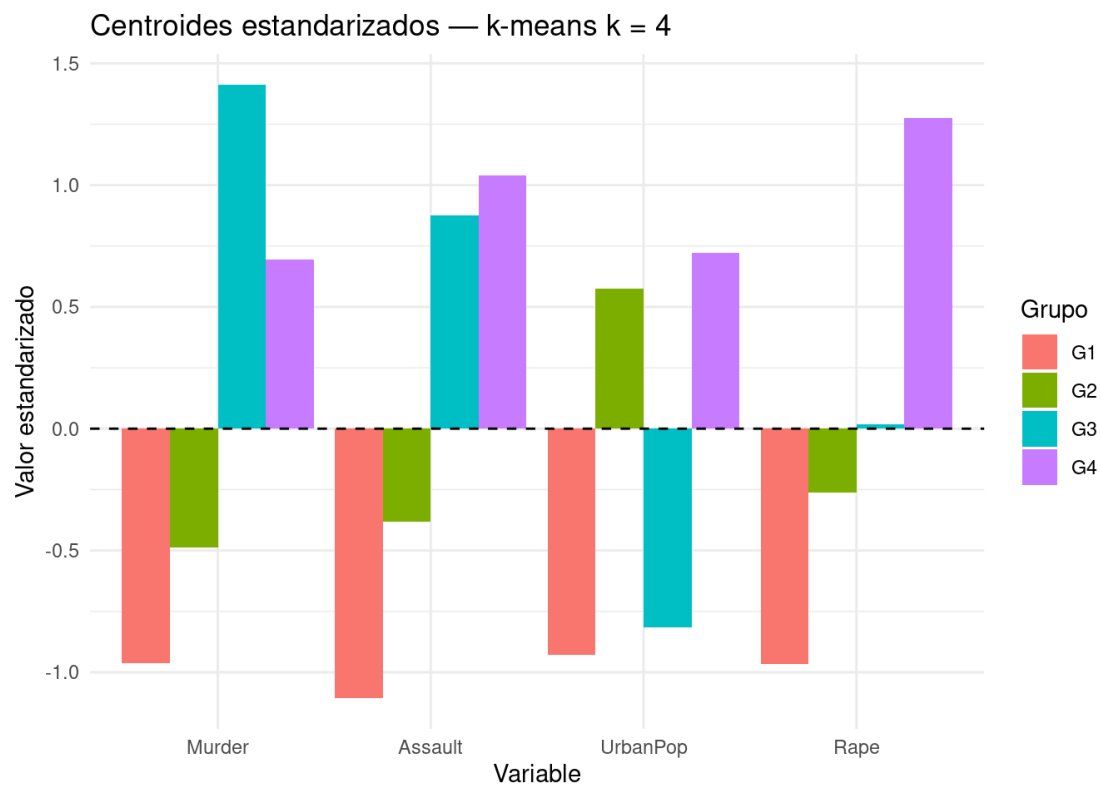
centroid_df
```

	Murder	Assault	UrbanPop	Rape	Grupo
1	-0.9615407	-1.1066010	-0.9301069	-0.96676331	G1
2	-0.4894375	-0.3826001	0.5758298	-0.26165379	G2
3	1.4118898	0.8743346	-0.8145211	0.01927104	G3
4	0.6950701	1.0394414	0.7226370	1.27693964	G4

```
centroid_long <- reshape(
  centroid_df,
  varying = names(centroid_df)[names(centroid_df) != "Grupo"],
  v.names = "Valor",
  timevar = "Variable",
  times = names(centroid_df)[names(centroid_df) != "Grupo"],
  direction = "long"
)

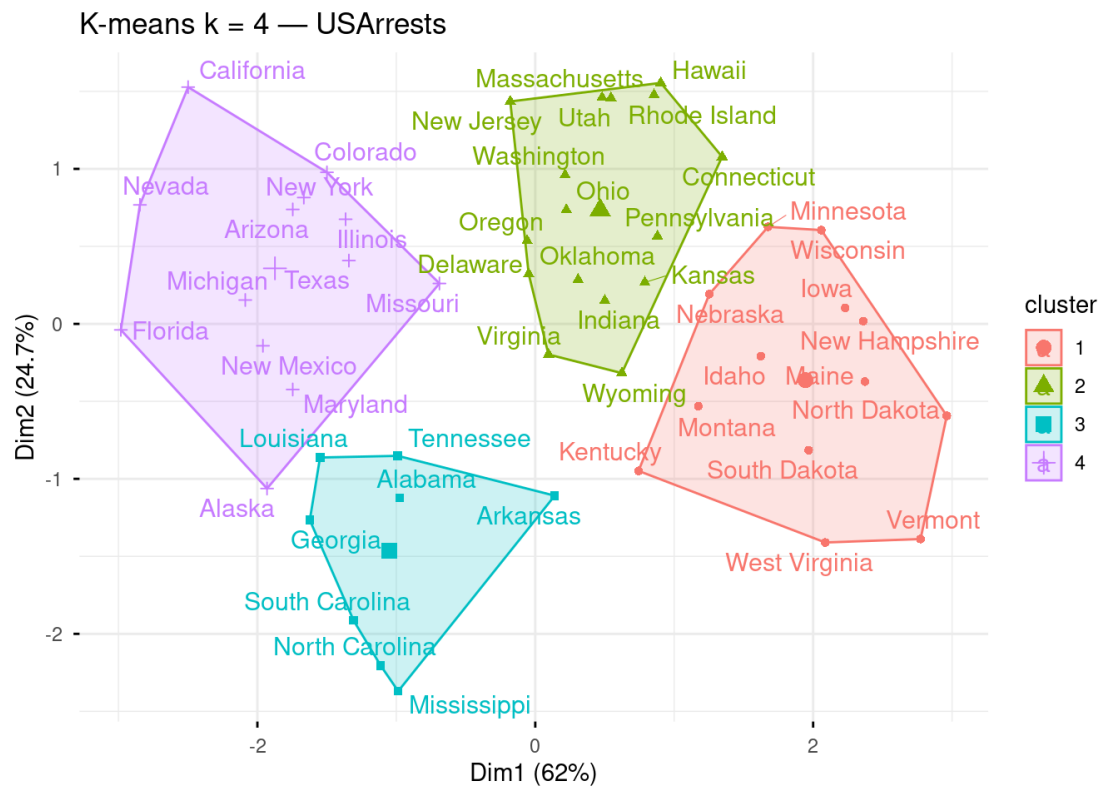
centroid_long$Variable <- factor(centroid_long$Variable,
                                levels = names(centroid_df)[names(centroid_df) != "Grupo"])

ggplot(centroid_long, aes(x = Variable, y = Valor, fill = Grupo)) +
  geom_col(position = "dodge") +
  geom_hline(yintercept = 0, linetype = "dashed") +
  theme_minimal() +
  labs(title = "Centroides estandarizados — k-means k = 4",
       y = "Valor estandarizado")
```



La línea horizontal en cero representa el promedio general.

```
fviz_cluster(km_final,
              data = USArrests_scale,
              ellipse.type = "convex",
              repel = TRUE,
              ggtheme = theme_minimal(),
              main = "K-means k = 4 — USArrests")
```



Validación y estabilidad

Un clustering puede verse bien, pero eso no alcanza. Queremos saber si los grupos son estables.

La pregunta es:

Si modificamos un poco los datos, ¿los mismos grupos reaparecen?

Usamos `clusterboot()`.

Bootstrap

```
library(fnc)
```

```
#####
```

```
set.seed(42)
```

```
boot_km <- clusterboot(USArrests_scale,  
                        B = 100,  
                        clustermethod = kmeansCBI,  
                        k = 4,  
                        nstart = 10,  
                        iter.max = 50,  
                        bootmethod = "boot")
```

```
boot 1  
boot 2  
boot 3  
boot 4  
boot 5  
boot 6  
boot 7  
boot 8  
boot 9  
boot 10  
boot 11  
boot 12  
boot 13  
boot 14  
boot 15  
boot 16  
boot 17  
boot 18  
boot 19  
boot 20  
boot 21  
boot 22  
boot 23  
boot 24  
boot 25  
boot 26  
boot 27  
boot 28  
boot 29  
boot 30  
boot 31  
boot 32  
boot 33  
boot 34  
boot 35  
boot 36  
boot 37
```


boot 38
boot 39
boot 40
boot 41
boot 42
boot 43
boot 44
boot 45
boot 46
boot 47
boot 48
boot 49
boot 50
boot 51
boot 52
boot 53
boot 54
boot 55
boot 56
boot 57
boot 58
boot 59
boot 60
boot 61
boot 62
boot 63
boot 64
boot 65
boot 66
boot 67
boot 68
boot 69
boot 70
boot 71
boot 72
boot 73
boot 74
boot 75
boot 76
boot 77
boot 78
boot 79
boot 80
boot 81
boot 82
boot 83
boot 84
boot 85

```
boot 86
boot 87
boot 88
boot 89
boot 90
boot 91
boot 92
boot 93
boot 94
boot 95
boot 96
boot 97
boot 98
boot 99
boot 100
```

```
print(boot_km)
```

```
* Cluster stability assessment *
Cluster method: kmeans
Full clustering results are given as parameter result
of the clusterboot object, which also provides further statistics
of the resampling results.
Number of resampling runs: 100

Number of clusters found in data: 4

Clusterwise Jaccard bootstrap (omitting multiple points) mean:
[1] 0.9360148 0.9268529 0.9384206 0.9228672
dissolved:
[1] 2 0 1 2
recovered:
[1] 90 96 91 93
```

Con bootstrap se remuestran filas con reposición. En este caso, estados. Algunos se repiten y otros quedan afuera. Luego se vuelve a correr k-means y se compara si los grupos originales reaparecen.

La comparación se hace con Jaccard:

```
Jaccard >= 0.75          = grupo muy estable
0.60 <= Jaccard < 0.75  = estabilidad moderada
Jaccard < 0.60          = grupo inestable
```

Submuestreo

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
```

```
set.seed(42)

sub_km <- clusterboot(USArrests_scale,
                      B = 100,
                      clustermethod = kmeansCBI,
                      k = 4,
                      nstart = 10,
                      iter.max = 50,
                      bootmethod = "subset",
                      subtuning = nrow(USArrests_scale) / 2)
```

```
subset 1
subset 2
subset 3
subset 4
subset 5
subset 6
subset 7
subset 8
subset 9
subset 10
subset 11
subset 12
subset 13
subset 14
subset 15
subset 16
subset 17
subset 18
subset 19
subset 20
subset 21
subset 22
subset 23
subset 24
subset 25
subset 26
subset 27
subset 28
subset 29
subset 30
subset 31
subset 32
subset 33
subset 34
subset 35
subset 36
subset 37
subset 38
```

subset 38
subset 39
subset 40
subset 41
subset 42
subset 43
subset 44
subset 45
subset 46
subset 47
subset 48
subset 49
subset 50
subset 51
subset 52
subset 53
subset 54
subset 55
subset 56
subset 57
subset 58
subset 59
subset 60
subset 61
subset 62
subset 63
subset 64
subset 65
subset 66
subset 67
subset 68
subset 69
subset 70
subset 71
subset 72
subset 73
subset 74
subset 75
subset 76
subset 77
subset 78
subset 79
subset 80
subset 81
subset 82
subset 83
subset 84
subset 85
subset 86

```
subset 87
subset 88
subset 89
subset 90
subset 91
subset 92
subset 93
subset 94
subset 95
subset 96
subset 97
subset 98
subset 99
subset 100
```

```
round(sub_km$subsetmean, 3)
```

```
[1] 0.872 0.868 0.876 0.891
```

```
round(boot_km$bootmean, 3)
```

```
[1] 0.936 0.927 0.938 0.923
```

El submuestreo toma una parte de las filas sin reposición. Es más exigente porque usa menos datos.

Ruido y jitter

```
set.seed(42)

noise_km <- clusterboot(USArrests_scale,
                        B = 100,
                        clustermethod = kmeansCBI,
                        k = 4,
                        nstart = 10,
                        iter.max = 50,
                        bootmethod = "noise")
```

```
noise 1
noise 2
noise 3
noise 4
noise 5
noise 6
noise 7
.
```

noise 8
noise 9
noise 10
noise 11
noise 12
noise 13
noise 14
noise 15
noise 16
noise 17
noise 18
noise 19
noise 20
noise 21
noise 22
noise 23
noise 24
noise 25
noise 26
noise 27
noise 28
noise 29
noise 30
noise 31
noise 32
noise 33
noise 34
noise 35
noise 36
noise 37
noise 38
noise 39
noise 40
noise 41
noise 42
noise 43
noise 44
noise 45
noise 46
noise 47
noise 48
noise 49
noise 50
noise 51
noise 52
noise 53
noise 54
noise 55
noise 56

```
noise 56  
noise 57  
noise 58  
noise 59  
noise 60  
noise 61  
noise 62  
noise 63  
noise 64  
noise 65  
noise 66  
noise 67  
noise 68  
noise 69  
noise 70  
noise 71  
noise 72  
noise 73  
noise 74  
noise 75  
noise 76  
noise 77  
noise 78  
noise 79  
noise 80  
noise 81  
noise 82  
noise 83  
noise 84  
noise 85  
noise 86  
noise 87  
noise 88  
noise 89  
noise 90  
noise 91  
noise 92  
noise 93  
noise 94  
noise 95  
noise 96  
noise 97  
noise 98  
noise 99  
noise 100
```

```
print(noise_km)
```

```
* Cluster stability assessment *
```

Cluster stability assessment

Cluster method: kmeans

Full clustering results are given as parameter result of the clusterboot object, which also provides further statistics of the resampling results.

Number of resampling runs: 100

Number of clusters found in data: 4

Clusterwise Jaccard replacement by noise mean:

```
[1] 0.8349346 0.8578561 0.8517460 0.8805081
```

dissolved:

```
[1] 4 1 15 4
```

recovered:

```
[1] 66 80 80 81
```

Noise reemplaza parte de los puntos por ruido.

```
set.seed(42)

jitt_km <- clusterboot(USArrests_scale,
  B = 100,
  clustermethod = kmeansCBI,
  k = 4,
  nstart = 10,
  iter.max = 50,
  bootmethod = "jitter")
```

```
jitter 1
jitter 2
jitter 3
jitter 4
jitter 5
jitter 6
jitter 7
jitter 8
jitter 9
jitter 10
jitter 11
jitter 12
jitter 13
jitter 14
jitter 15
jitter 16
jitter 17
jitter 18
jitter 19
jitter 20
jitter 21
```


jitter 21
jitter 22
jitter 23
jitter 24
jitter 25
jitter 26
jitter 27
jitter 28
jitter 29
jitter 30
jitter 31
jitter 32
jitter 33
jitter 34
jitter 35
jitter 36
jitter 37
jitter 38
jitter 39
jitter 40
jitter 41
jitter 42
jitter 43
jitter 44
jitter 45
jitter 46
jitter 47
jitter 48
jitter 49
jitter 50
jitter 51
jitter 52
jitter 53
jitter 54
jitter 55
jitter 56
jitter 57
jitter 58
jitter 59
jitter 60
jitter 61
jitter 62
jitter 63
jitter 64
jitter 65
jitter 66
jitter 67
jitter 68
jitter 69

```
~
jitter 70
jitter 71
jitter 72
jitter 73
jitter 74
jitter 75
jitter 76
jitter 77
jitter 78
jitter 79
jitter 80
jitter 81
jitter 82
jitter 83
jitter 84
jitter 85
jitter 86
jitter 87
jitter 88
jitter 89
jitter 90
jitter 91
jitter 92
jitter 93
jitter 94
jitter 95
jitter 96
jitter 97
jitter 98
jitter 99
jitter 100
```

```
print(jitt_km)
```

```
* Cluster stability assessment *
```

```
Cluster method: kmeans
```

```
Full clustering results are given as parameter result  
of the clusterboot object, which also provides further statistics  
of the resampling results.
```

```
Number of resampling runs: 100
```

```
Number of clusters found in data: 4
```

```
Clusterwise Jaccard jittering mean:
```

```
[1] 1 1 1 1
```

```
dissolved:
```

```
[1] 0 0 0 0
```

```
recovered:
```

```
recovered.  
[1] 100 100 100 100
```

Jitter mueve levemente los puntos originales.

Comparación general

```
estabilidad <- c(  
  boot    = mean(boot_km$bootmean),  
  subset  = mean(sub_km$subsetmean),  
  noise   = mean(noise_km$noisemean),  
  jitter  = mean(jitt_km$jittermean)  
)  
  
round(estabilidad, 3)
```

```
boot subset noise jitter  
0.931  0.877  0.856  1.000
```

Valores altos indican que la solución es estable frente a distintos tipos de perturbación.

Comparar k-means con jerárquico

Podemos comparar dos particiones con ARI.

```
library(mclust)
```

Package 'mclust' version 6.1.2
Type 'citation("mclust")' for citing this R package in publications.

```
set.seed(42)  
  
km4 <- kmeans(USArrests_scale,  
             centers = 4,  
             nstart = 25)  
  
hier4 <- cutree(hc_ward, k = 4)  
  
adjustedRandIndex(km4$cluster, hier4)
```

```
[1] 0.8244826
```

ARI responde:

■

¿Las mismas observaciones quedaron agrupadas juntas en ambos métodos?

ARI = 1 = coincidencia perfecta

ARI = 0 = no mejor que azar

ARI < 0 = peor que azar

Variantes

Hasta ahora vimos jerárquico y k-means. Ahora miramos variantes para entender otras posibilidades.

Usamos `iris`, porque conocemos sus tres especies reales.

```
data(iris)
iris_scale <- scale(iris[, 1:4])
```

PAM

PAM es parecido a k-means, pero usa medoides.

k-means = centroides, puntos promedio

PAM = medoides, observaciones reales

```
pam_iris <- pam(iris_scale,
               k = 3,
               metric = "euclidean")

pam_iris$medoids
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
[1,]	-1.0184372	0.7861738	-1.2791040	-1.3110521
[2,]	1.1553023	-0.1315388	0.9868021	1.1816087
[3,]	-0.1730941	-0.5903951	0.4203256	0.1320673

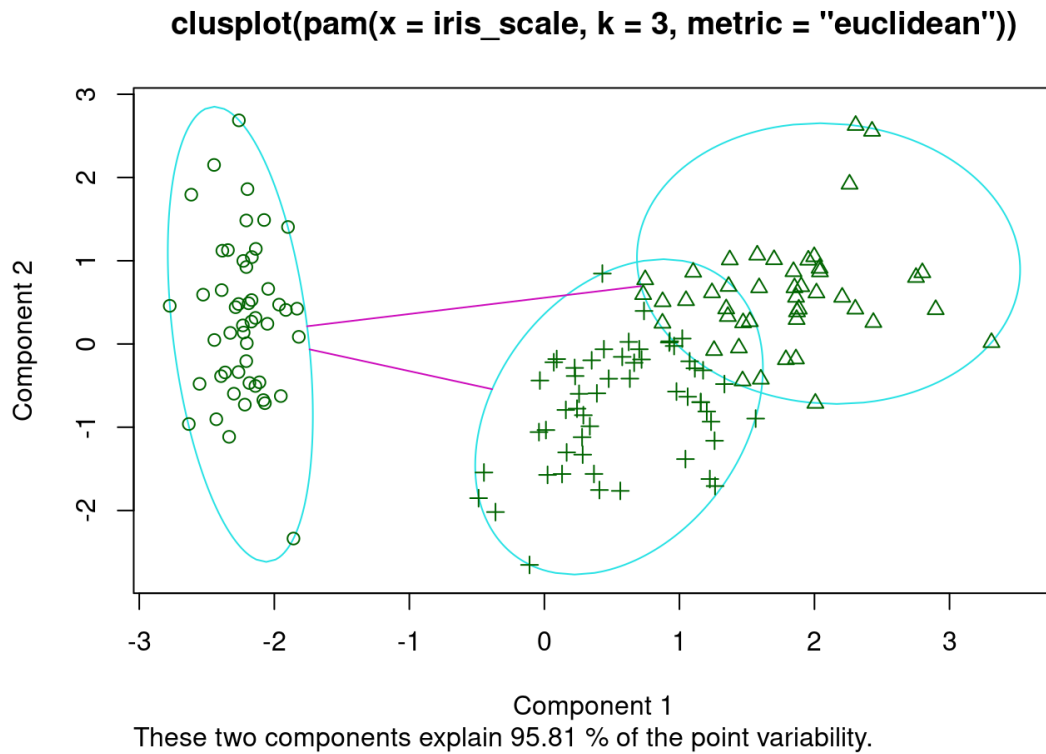
```
pam_iris$id.med
```

```
[1] 8 113 56
```

```
pam_iris$silinfo$avg.width
```

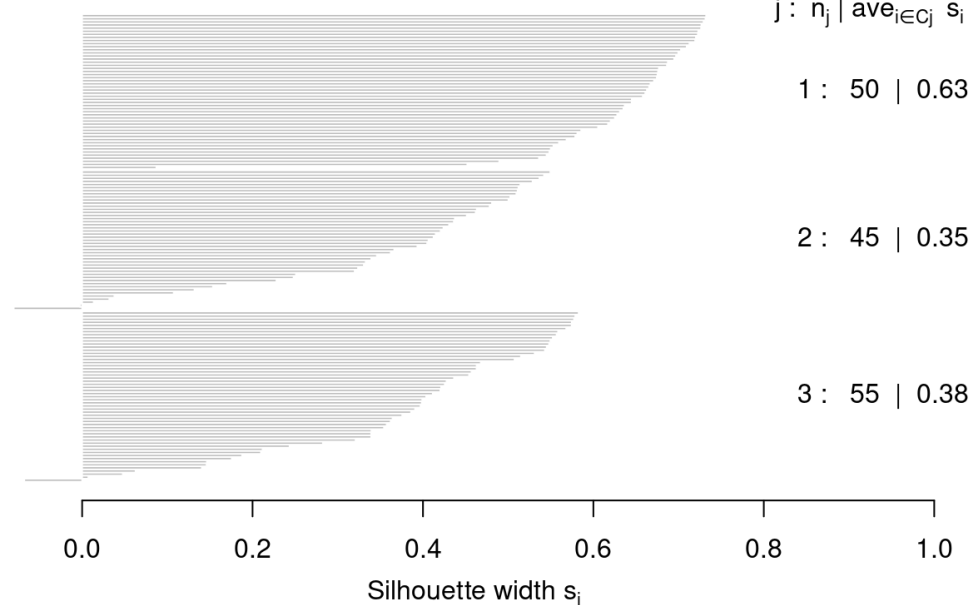
```
[1] 0.4566432
```

```
plot(pam_iris)
```



Silhouette plot of pam(x = iris_scale, k = 3, metric = "euclidean")

n = 150



Average silhouette width : 0.46

```
adjustedRandIndex(pam_iris$clustering, iris$Species)
```

```
[1] 0.6416027
```

PAM suele ser más robusto que k-means frente a outliers, porque usa observaciones reales como representantes.

AGNES

AGNES es clustering jerárquico aglomerativo.

empieza separado → va uniendo

```
agnes_iris <- agnes(iris_scale, method = "ward")
agnes_iris$ac
```

```
[1] 0.9867558
```

El coeficiente aglomerativo resume qué tan marcada es la estructura jerárquica.

```
metodos <- c("average", "single", "complete", "ward")

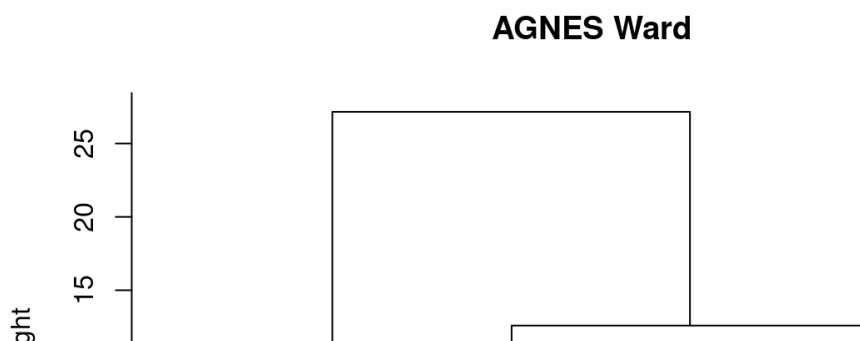
ac_vals <- sapply(metodos, function(m) {
  agnes(iris_scale, method = m)$ac
})

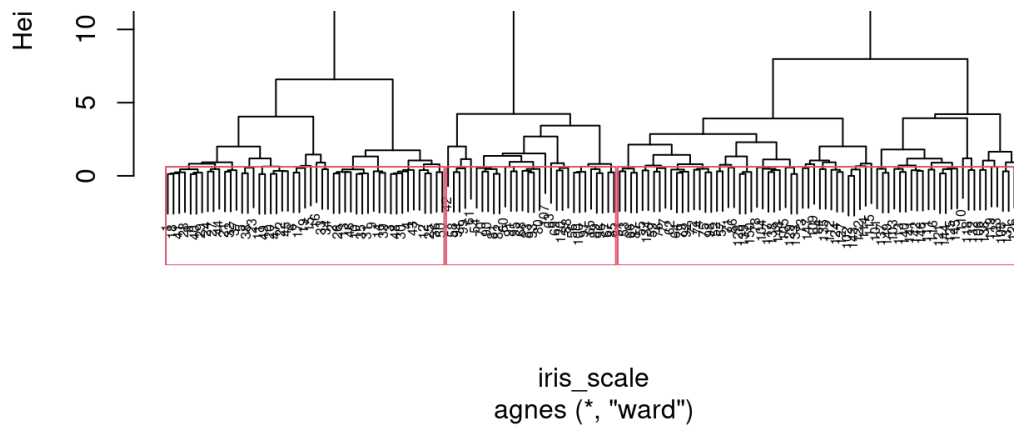
ac_vals
```

```
      average      single  complete      ward
0.9035705 0.8023794 0.9438858 0.9867558
```

```
pltree(agnes_iris,
       cex = 0.5,
       main = "AGNES Ward")

rect.hclust(agnes_iris, k = 3)
```





CLARA

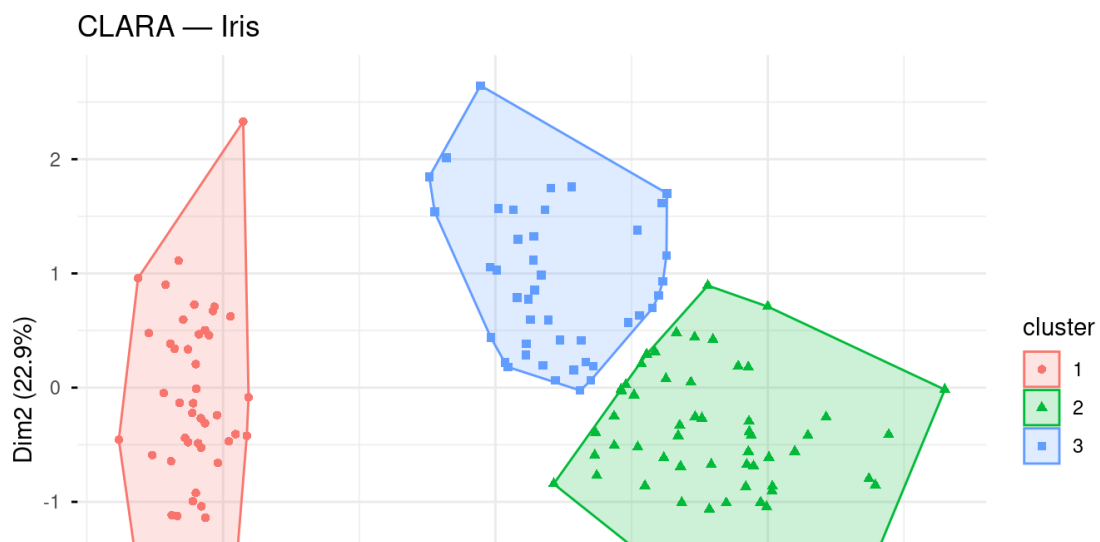
CLARA es una versión escalable de PAM. Usa muestras del dataset y conserva la mejor solución.

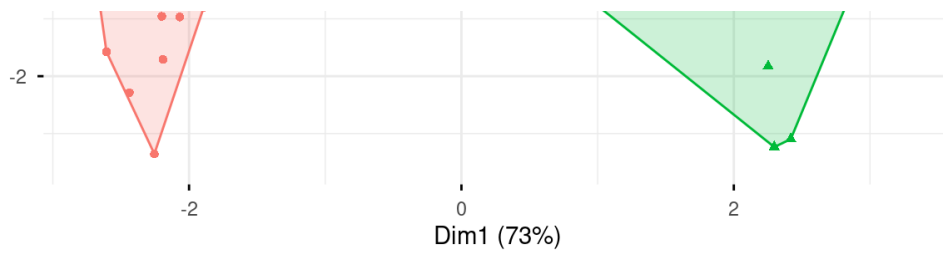
```
clara_iris <- clara(iris_scale,
  k = 3,
  samples = 50,
  metric = "euclidean")
```

```
clara_iris$silinfo$avg.width
```

```
[1] 0.4867686
```

```
fviz_cluster(clara_iris,
  geom = "point",
  ggtheme = theme_minimal(),
  main = "CLARA — Iris")
```





DIANA

DIANA es clustering jerárquico divisivo.

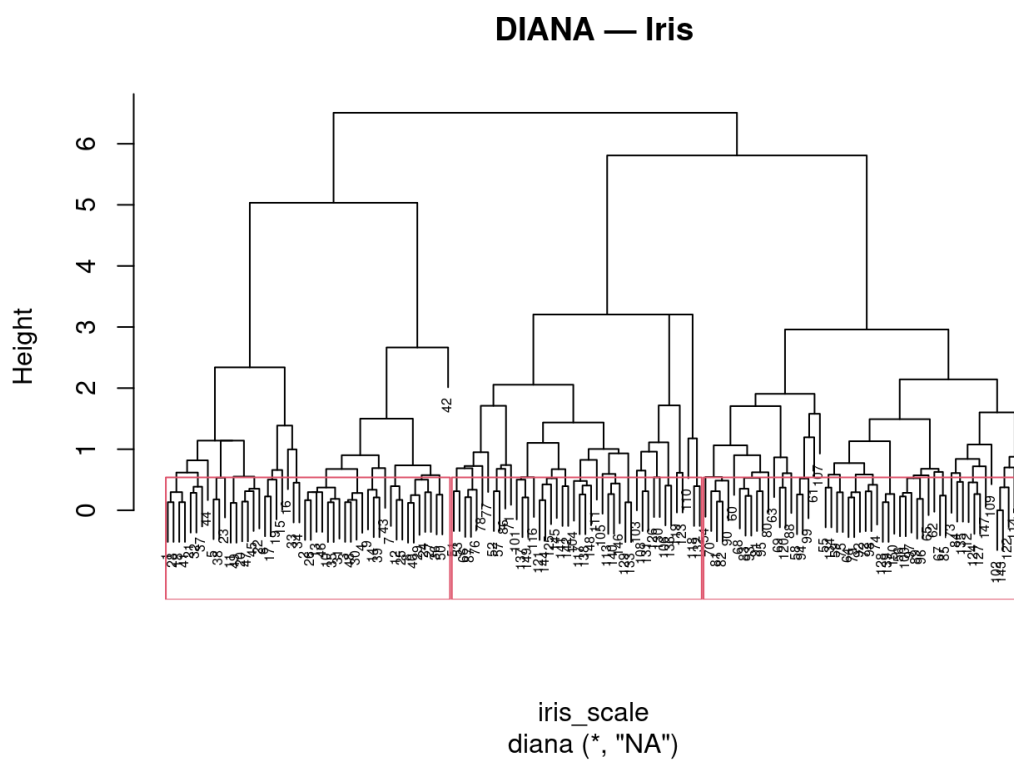
empieza todo junto → va separando

```
diana_iris <- diana(iris_scale)
diana_iris$dc
```

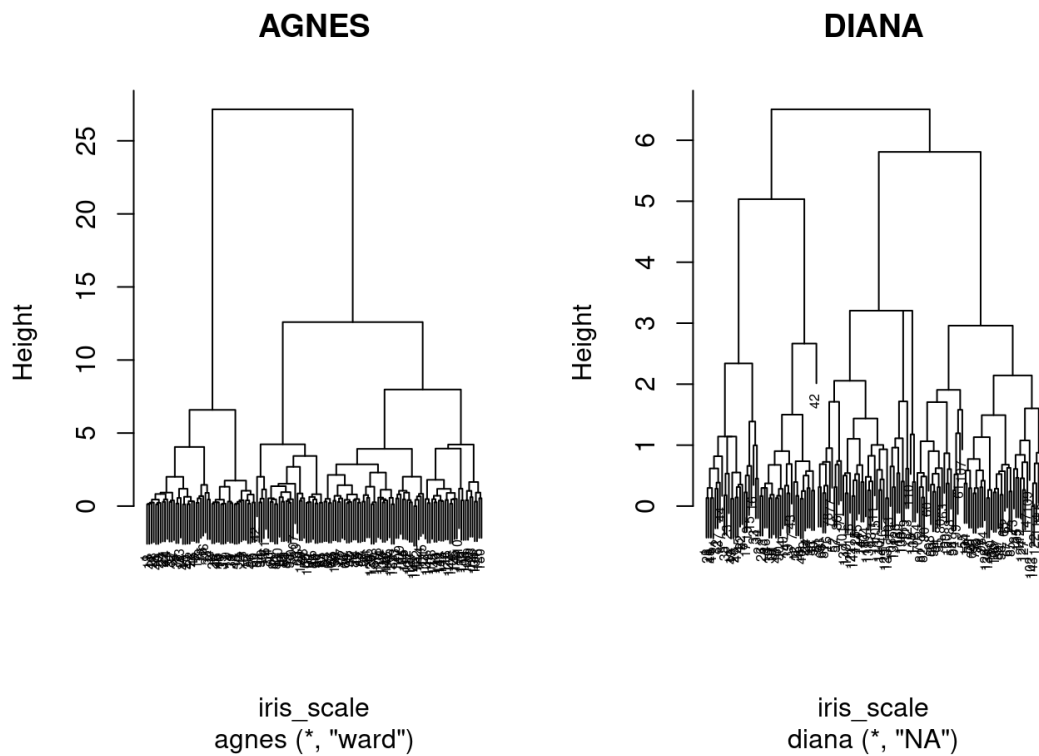
```
[1] 0.9397208
```

```
pltree(diana_iris,
       cex = 0.5,
       main = "DIANA - Iris")

rect.hclust(diana_iris, k = 3)
```




```
par(mfrow = c(1, 2))
pltree(agnes_iris, cex = 0.5, main = "AGNES")
pltree(diana_iris, cex = 0.5, main = "DIANA")
```



```
par(mfrow = c(1, 1))
```

FANNY

FANNY es clustering difuso. No asigna cada observación a un único grupo de forma rígida. Estima grados de pertenencia.

```
fanny_iris <- fanny(iris_scale,
                    k = 3,
                    metric = "euclidean")

head(fanny_iris$membership)
```

```
      [,1]      [,2]      [,3]
[1,] 0.8385968 0.07317705 0.08822614
[2,] 0.6704536 0.14154547 0.18800092
[3,] 0.7708430 0.10070647 0.12845058
```

```
[4,] 0.7086764 0.12665561 0.16466796
[5,] 0.8047709 0.08938752 0.10584154
[6,] 0.6200074 0.18062227 0.19937035
```

```
fanny_iris$coeff
```

```
dunn_coeff normalized
0.4752894 0.2129341
```

```
fanny_iris$silinfo$avg.width
```

```
[1] 0.4566338
```

Cada fila de **membership** suma 1. Si una observación tiene pertenencias repartidas, es una observación ambigua o de frontera.

```
cluster_hard <- fanny_iris$clustering
adjustedRandIndex(cluster_hard, iris$Species)
```

```
[1] 0.6410059
```

```
prob_max <- apply(fanny_iris$membership, 1, max)

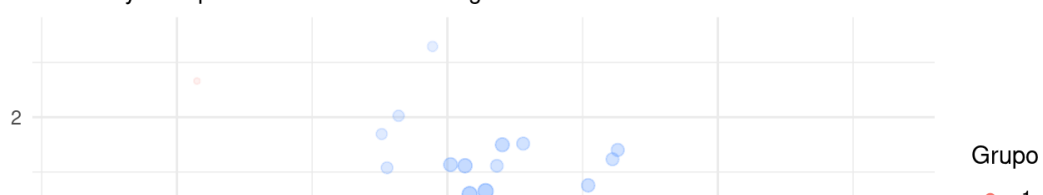
pca_iris <- prcomp(iris_scale)

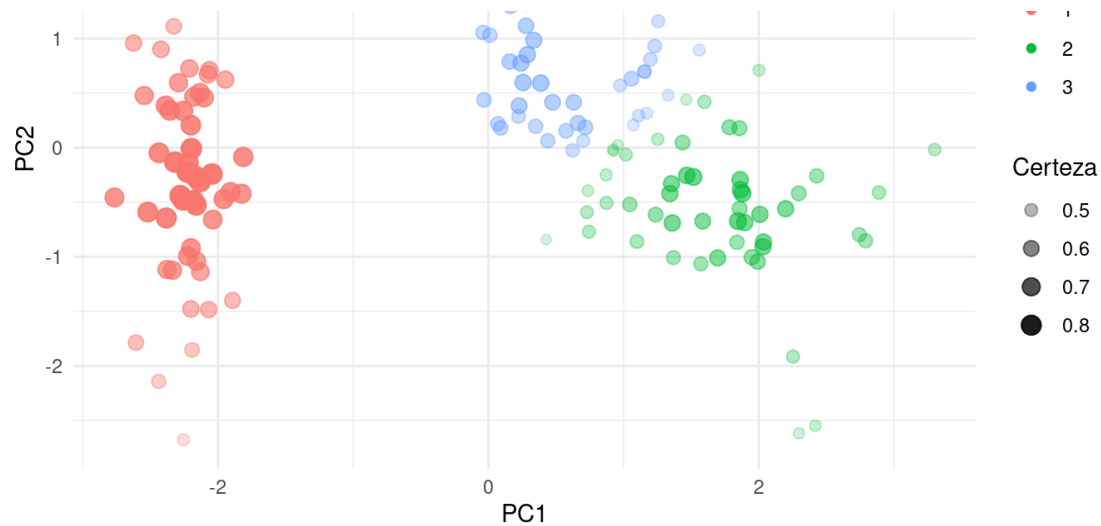
df_fanny <- data.frame(
  PC1 = pca_iris$x[, 1],
  PC2 = pca_iris$x[, 2],
  Grupo = as.factor(cluster_hard),
  Certeza = prob_max
)

ggplot(df_fanny, aes(PC1, PC2, color = Grupo, size = Certeza, alpha = Certeza)) +
  geom_point() +
  scale_size_continuous(range = c(1, 4)) +
  theme_minimal() +
  labs(title = "FANNY - Iris",
       subtitle = "Tamaño y transparencia = certeza de asignación")
```

FANNY — Iris

Tamaño y transparencia = certeza de asignación





Limitaciones de k-means

K-means funciona bien cuando los grupos son compactos, separados y de tamaño parecido.

Puede fallar con:

- formas no esféricas
- grupos de tamaños muy diferentes
- outliers
- densidades distintas

Formas no esféricas

```
set.seed(42)

theta <- seq(0, 2 * pi, length.out = 150)

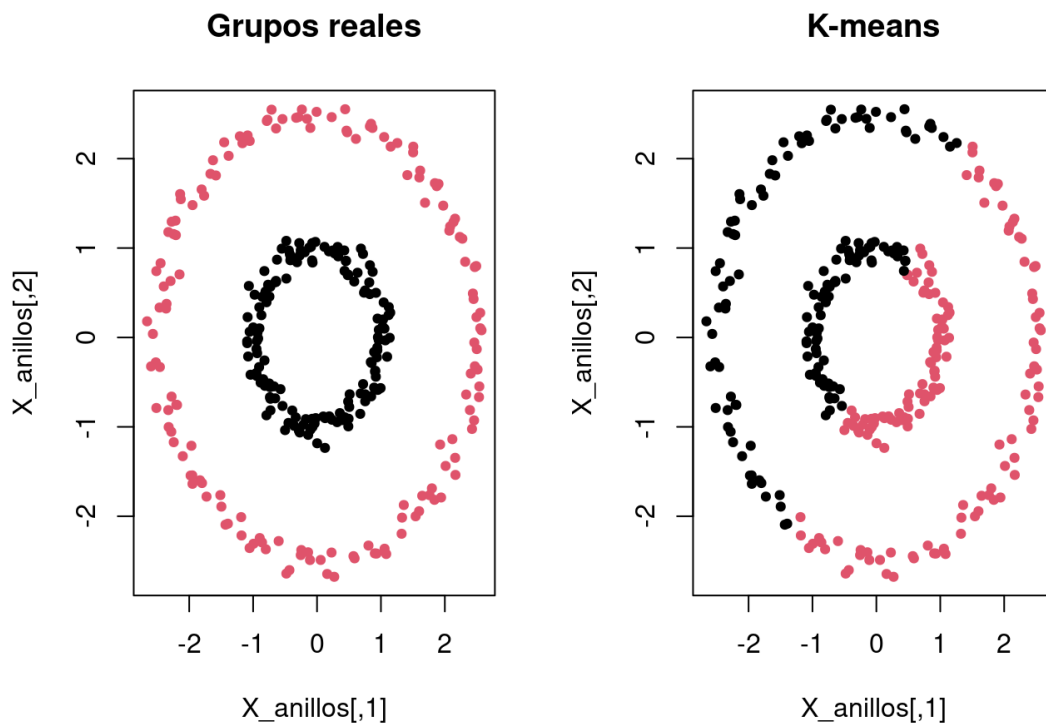
anillo_int <- cbind(
  cos(theta) * 1 + rnorm(150, 0, 0.1),
  sin(theta) * 1 + rnorm(150, 0, 0.1)
)

anillo_ext <- cbind(
  cos(theta) * 2.5 + rnorm(150, 0, 0.1),
  sin(theta) * 2.5 + rnorm(150, 0, 0.1)
)

X_anillos <- rbind(anillo_int, anillo_ext)
real_anillos <- c(rep(1, 150), rep(2, 150))
```

```
km_anillos <- kmeans(X_anillos, centers = 2, nstart = 25)
```

```
par(mfrow = c(1, 2))
plot(X_anillos, col = real_anillos, pch = 19, cex = 0.7,
     main = "Grupos reales")
plot(X_anillos, col = km_anillos$cluster, pch = 19, cex = 0.7,
     main = "K-means")
```



```
par(mfrow = c(1, 1))
```

Los grupos reales son dos anillos. K-means no recupera esa forma porque busca grupos compactos alrededor de centroides.

Tamaños muy diferentes

```
set.seed(42)

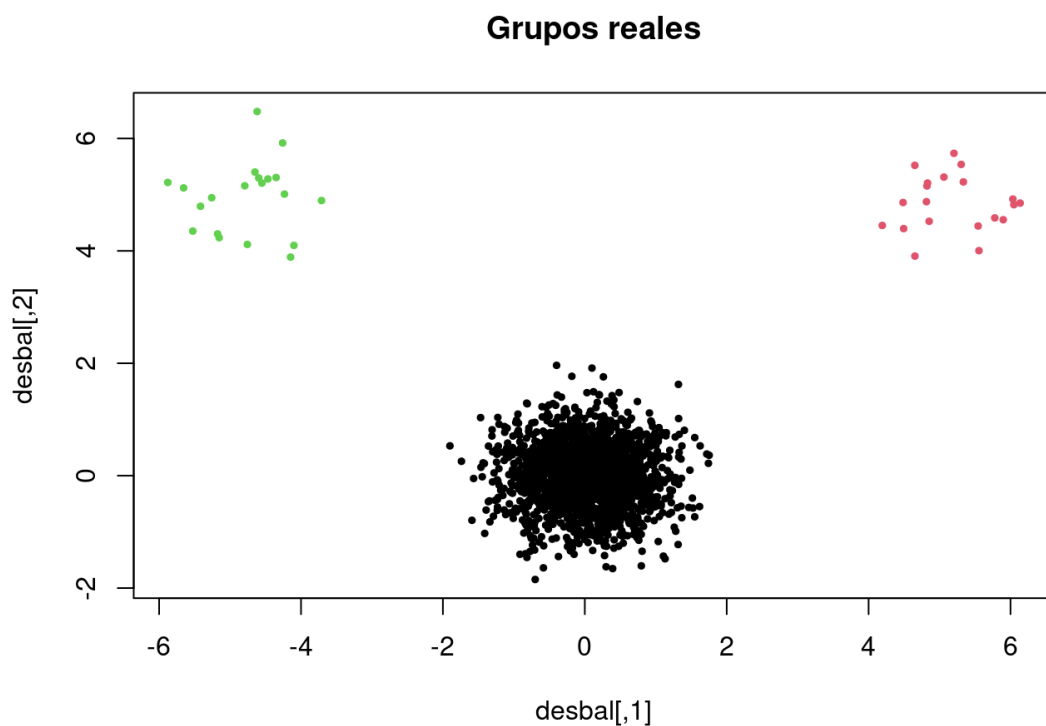
desbal <- rbind(
  mvrnorm(2000, mu = c(0, 0), Sigma = diag(0.3, 2)),
  mvrnorm(20, mu = c(5, 5), Sigma = diag(0.3, 2)),
  mvrnorm(20, mu = c(-5, 5), Sigma = diag(0.3, 2))
)
```

```
real_desbal <- c(rep(1, 2000), rep(2, 20), rep(3, 20))
km_desbal <- kmeans(desbal, centers = 3, nstart = 25)
```

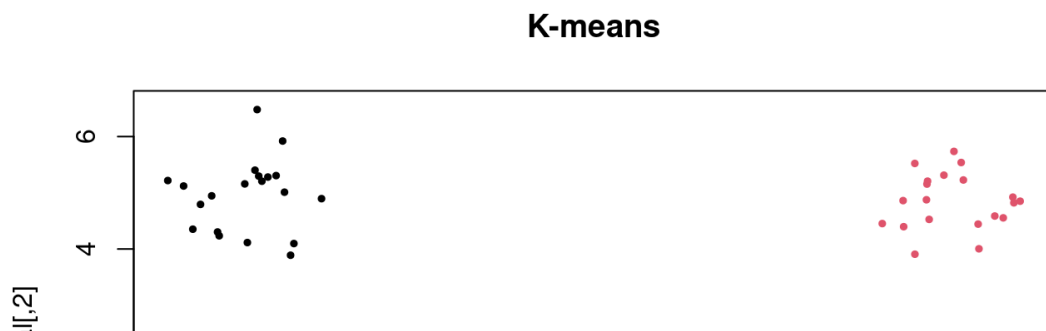
```
adjustedRandIndex(km_desbal$cluster, real_desbal)
```

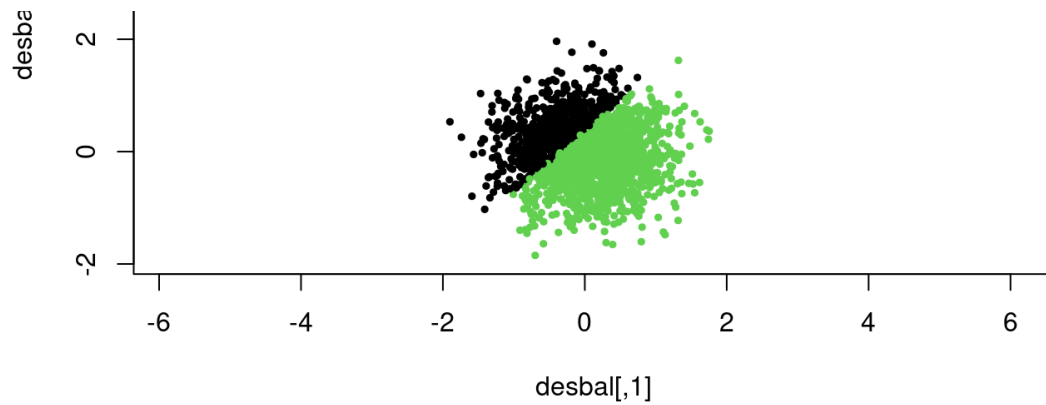
```
[1] 0.06670056
```

```
plot(desbal, col = real_desbal, pch = 19, cex = 0.5,
     main = "Grupos reales")
```



```
plot(desbal, col = km_desbal$cluster, pch = 19, cex = 0.5,
     main = "K-means")
```





El algoritmo puede dividir el grupo grande o absorber mal los grupos chicos.

Outliers

```
set.seed(42)

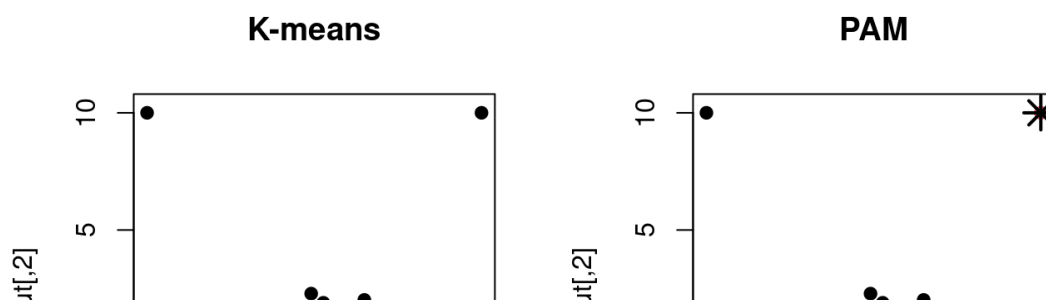
data_con_out <- mvrnorm(50, mu = c(0, 0), Sigma = diag(1, 2))
data_con_out <- rbind(
  data_con_out,
  matrix(c(10, 10, -10, 10, 10, -10), ncol = 2, byrow = TRUE)
)

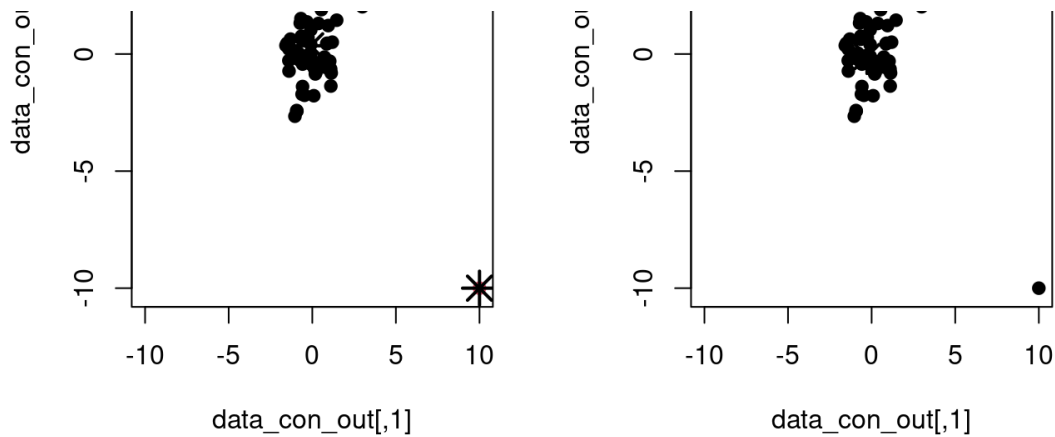
km_out <- kmeans(data_con_out, centers = 2, nstart = 25)
pam_out <- pam(data_con_out, k = 2)
```

```
par(mfrow = c(1, 2))

plot(data_con_out, col = km_out$cluster, pch = 19,
     main = "K-means")
points(km_out$centers, pch = 8, cex = 2, lwd = 2)

plot(data_con_out, col = pam_out$clustering, pch = 19,
     main = "PAM")
points(data_con_out[pam_out$id.med, ], pch = 8, cex = 2, lwd = 2)
```





```
par(mfrow = c(1, 1))
```

PAM suele ser más robusto porque usa medoides, es decir, observaciones reales, no promedios arrastrados por valores extremos.

Cierre

K-means no está mal. Es simple, rápido y útil cuando los grupos son compactos, separados y de tamaño parecido.

Pero si los grupos tienen formas complejas, densidades distintas, tamaños muy desbalanceados u outliers, conviene probar otros métodos.

Ese es el puente hacia el próximo bloque:

GMM = grupos como nubes gaussianas
 DBSCAN = grupos como zonas densas conectadas
 OPTICS = estructura de densidad a distintas escalas

Ejercicios sugeridos

Ejercicio 1 — wine

1. Escalar variables numéricas.
2. Calcular distancias y comparar linkages.
3. Elegir `k` y cortar dendrograma.
4. Calcular silueta.
5. Comparar grupos con regiones de origen.

Ejercicio 2 — crabs

1. Justificar escalado.
2. Comparar k-means con `nstart = 1` y `nstart = 25`.
3. Elegir `k` con codo y silueta.
4. Interpretar centroides.
5. Comparar con especie y sexo usando ARI.

Ejercicio 3 — estabilidad en crabs

1. Aplicar `clusterboot()` con bootstrap.
2. Aplicar reemplazo por ruido.
3. Aplicar jitter.
4. Comparar estabilidad de los métodos.